

EOS平台-业务-引擎规范

概述：定义 EOS 平台的 UI 交互模型（含六种页面类型）、输出文档模型（正式/轻量文档、封面-正文 1:1 组装）、引擎分层体系与 CU 配置单元模型、操作页面组件与构件体系、三类产品（A1/A2/Bn）、资产治理业务，以及三路径（biz/eng/nfr）自闭环 Skill 的编制模式和职责定位。

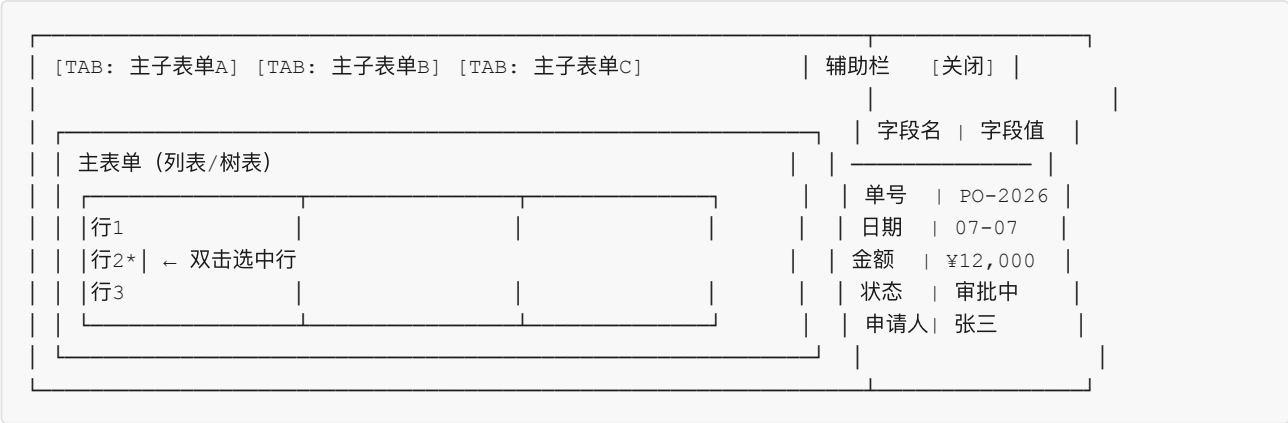
版本：v4.8 | **修订：**2026-07-11 | **说明：**§七~§九 全章重构（产出物链清晰化 + 双表合一 + NFR→eng 单向流 + 平台服务组件）+ 附录A 重构为四部分结构 + §10.4/§10.6 参数对齐 + §十一 新增产品文档结构、读写及反馈（AI可以处理节点、子条目状态、变更感知、HEAD 追踪）+ 附录A 读取/写回协议全量同步 + 全文缺陷修复。

一、平台界面与业务场景

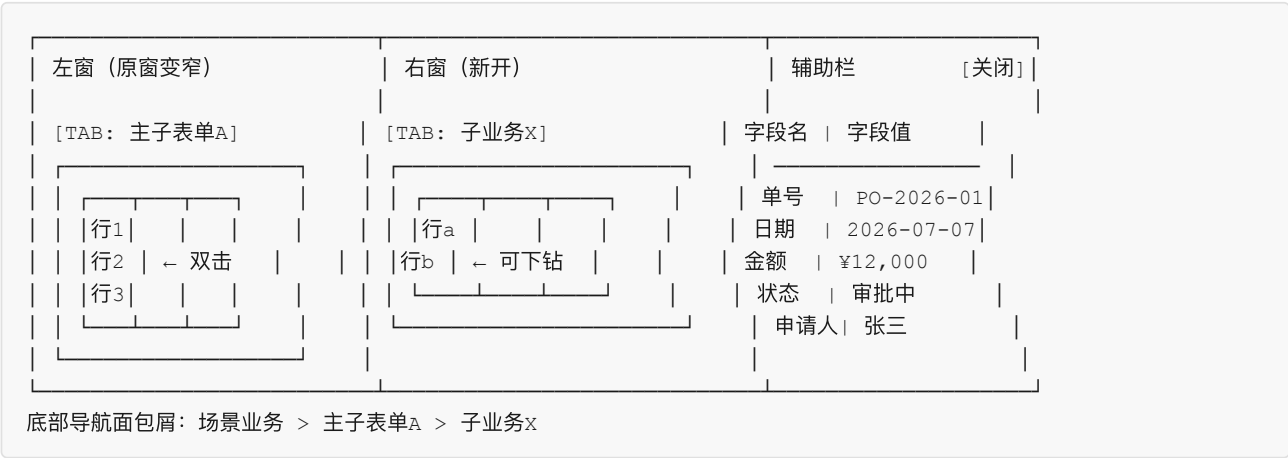
1.1 平台UI

用户单击菜单引擎发布的末级菜单进入场景业务，运行时 UI 遵循以下交互模型：

初始态：浏览器窗口为单窗全宽，顶部多 TAB 对应场景业务下的多个主子表单；选中一行数据双击可下钻。初始态无底部导航面包屑。右侧辅助栏位展开，以两列（字段名 | 字段值）显示选中行的详细数据。



分屏态（下钻触发）：双击主表单行数据 → 窗口分裂为左右双窗。左窗为原主子表单（变窄），右窗显示选中行数据的子表单；右窗 TAB 对应子业务，每个关联实体占一个 TAB。右侧辅助栏位展开时以两列（字段名 | 字段值）显示选中行详情。



持续下钻：在右窗双击行 → 右窗移至左窗，新子业务 TAB 出现在右窗位置。可一直下钻直至系统下钻层级数限制。底部常驻面包屑导航，双击任意节点可跳转。

交互规则：

规则	说明
初始单窗	进入场景业务时为单窗+右侧辅助栏展开，顶部多TAB；不下钻不分屏
主子表单 TAB	顶部 TAB 对应该场景业务下的多个主子表单
首次下钻	在主表单双击某一行数据触发分屏，右窗显示子表单数据
递归下钻	右窗→左窗，新右窗出现，直到系统层级数限制
面包屑导航	底部常驻，显示当前层级路径；双击任意节点跳转至该层级
子表单显示规则	功能引擎基于主表单行数据内容匹配规则，动态确定展示子表单集合
辅助栏展开态	右栏展开，以两列（字段名 字段值）显示选中行详情；主内容区向左缩
辅助栏切换	展开态下切换选中行，辅助栏自动更新为新的行详情
辅助栏关闭操作	单击"关闭"按钮，辅助栏关闭，按钮显示"编辑"；主内容区恢复全宽
辅助栏展开操作	单击"编辑"按钮，辅助栏展开；分屏态下辅助栏展开时内容区向左缩

1.2 页面类型

功能表单对用户呈现的页面类型仅为以下六种有限模式，每种模式描述功能表单在窗口中的布局方式和交互范围：

序号	页面类型	描述	适用场景
①	单窗列表+辅助栏	顶部 TAB，主区域为数据列表（表格），右侧为辅助栏位	纯列表管理，如实体数据维护
②	单窗树表+辅助栏	顶部 TAB，主区域为树形结构表，右侧为辅助栏位	树形数据管理，如分类/BOM 管理
③	单窗单据页	顶部 TAB，主区域为单据页（字段+操作按钮），可选配右侧辅助栏	独立录入/编辑/审批页
④	左窗主表+右窗子表+辅助栏	分屏模式，左窗为列表/树表，右窗为对应子业务数据，辅助栏在右侧	1:N 主子数据管理
⑤	全屏单据页	无顶部 TAB，整页为一张单据页，可选配右侧辅助栏	大篇幅录入/展示，如合同编辑
⑥	弹窗	模态弹出页面，非独立 TAB	临时焦点操作，如选择数据

1.3 辅助栏

辅助栏是右侧可展开信息面板，可出现在任意页面类型上（①~⑤），行为在所有页面类型上保持一致：

关闭态：

- 按钮标签显示"编辑"
- 主内容区（单窗或双窗）全宽显示

展开态（单击"编辑"或选中行后）：

- 主内容区向左缩，辅助栏占据右边空间
- 辅助栏以两列单据页方式显示选中数据行的详情——第一列字段名称，第二列字段值
- 当用户在主内容区选择另一行数据时，辅助栏自动切换显示新选中行的详细信息

关闭（单击"关闭"按钮）：

- 辅助栏关闭
- 主内容区向右展开至屏幕边，恢复全宽

1.4 平台菜单

菜单引擎（@engine-menu）生成并维护 EOS 平台的多级菜单树，分为两类节点：

- **非末级节点**：定义所属父节点、显示顺序、可见性和可达性权限——仅承载分组和导航。
- **末级节点**：定义所包含的主子表单部件序列（即**场景业务**）、显示顺序和依赖关系，以及该节点的可见性和可达性权限。

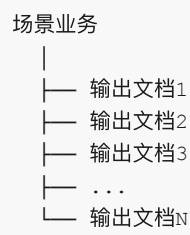
菜单视角（ML）自上而下为：M0 平台 → M1 一级菜单 → M2 二级菜单 → M3 三级菜单 → M4 末级菜单（→SL5 场景业务）。菜单引擎对每条场景业务无条件支撑，无需判定。

权限归属：菜单节点对角色/人员的可见或不可见规则，由菜单引擎承载。物理表数据行或列对角色/人员的权限，由功能引擎承载。

1.5 业务场景

场景业务是 STR-F（相关方功能需求架构末级节点）的组织单元。每条场景业务定义一条**端到端业务闭环**——从需求提出到需求被满足并确认的完整链路。

场景业务内部为一组有先后流转依赖的**输出文档序列**：



在 EOS 平台上，场景业务（SL5）由菜单引擎的末级节点编排支撑——末级菜单所包含的功能表单/主子表单序列即为场景业务的运行时形态。

场景业务的类型由 AI 根据 OR 语义判定：

类型	识别词	闭环标准
执行类（D）	创建/提交/审批/交付/验收等动作词	D 闭合——从需求提出到需求满足
管理类（PCA）	计划/监控/分析/纠正/识别等动作词	PCA 闭合——计划→监控→分析→纠正

D 和 PCA 仅在正文数据类型上有差异（产品数据 vs 管理数据），在 wft01/02/03-biz 的处理逻辑上**完全同构**——都是场景业务→输出文档序列→每个文档有封面+正文+任务+引用。区别在于：D 类输出文档的正文是产品数据（物料/价格/检验值），文档间流转以输入-输出传递链为主；PCA 类输出文档的正文是管理数据（指标定义/看板布局/计划任务/风险描述/问题记录），任务类型=计划的文档发布后通过任务关系触发流程/工单。类型判定仅影响 wft02-biz 的引擎推断偏好和 wft03-biz 的配置建议，不影响处理流程结构。

输出文档的完整模型见 §二。

二、输出文档模型

输出文档（PL5） 是场景业务的组成单元——每条场景业务由一组有先后流转依赖的输出文档序列定义。输出文档是 EOS 平台的核心信息载体，其模型回答三个基础问题：文档是什么、由什么组成、如何流转。

核心结论：文档不是引擎概念，而是**实体组装模式**——封面实体（DocumentCover 表一行，全系统一张表）+ 正文实体（1:1）+ 支持信息（→其他文档的多对多引用）+ 处理配置（wft02-biz 定义任务节点，wft03-biz 生成引擎 CU 配置）。封面和正文都是普通实体，由实体引擎生成物理表；正文被各引擎按需转化为功能表单，封面由功能引擎的表单能力转化为封面表单，两者由功能引擎的组装能力按 1:1 主子关系组装。因此**不设独立的"文档引擎"**——文档是实体组装模式，而非独立引擎概念。台账类业务由台账引擎（@engine-ledger）支撑——将物理表转换为台账功能表单并按主子关系装配为主子功能表单（详见 §3.2）。

2.1 封面判定：正式文档与轻量文档

文档是否需要封面，取决于它是否有**独立于正文 CRUD 的生命周期**——需要审批流转、版本追溯、派发验收或独立状态变迁的文档，才有封面。

正式文档（有封面，有独立生命周期）	轻量文档（无封面，正文即文档）
DocumentCover 表一行 ├— 单据号/标题 ├— 状态/版本 ├— 创建人/部门/日期 ├— 处理配置（可选的流程/工单/计划） └— 1:1 → 正文实体（1 行 = 1 文档内容条目实例） 例子： 采购申请单/检验记录/不合格品报告 项目立项书/设备维修工单 指标文档（独立维护，有审批/版本） 看板文档（独立维护，有审批/版本）	无封面行 正文实体 = 文档本身 1 行 = 1 文档内容条目实例 可选的任务类型（流程/工单/计划）—— 任务状态由对应引擎在正文实体上直接管理 例子： 会议纪要（流程审批）/技术规范条目/ 参考文档/经验教训/公告通知

2.2 正式文档的组成

正式文档 (PL5)

- 封面实体 (DocumentCover 表一行)
 - 记载单据号/标题/状态/版本/创建人/部门/日期等文档级元数据
 - 由功能引擎的表单能力转化为封面功能表单 (单据头样式)
- 正文实体 (1:1 绑定封面行, 1 行 = 1 文档内容条目实例)
 - 正文即实体——该文档承载的业务数据内容
 - 由对应引擎转化为功能表单, 形态根据业务而定:
 - 台账 (列表/树表)、指标 (列表+图示 2×3)、看板 (视图/卡片/泳道)、
 - 项目 (阶段/里程碑)、WBS (树形分解)、计划 (甘特/任务)、
 - 流程 (审批流转)、工单 (派发/处理/验收)
 - 同一正文可被多个引擎转化, 互不冲突
- 支持信息 (→其他文档的多对多引用)
 - 被引用目标是正式文档时指向 DocumentCover 行, 轻量文档时指向正文实体行
 - 引用关系带角色标签 (输入文档/规范文档/指南文档/合规文档/资源文档)
- 处理配置 (wft02-biz 定义任务节点, wft03-biz 生成引擎 CU 配置)
 - 封面行被流程/工单/计划引擎驱动流转的节点序列
 - 每个节点定义: 处理角色、可执行动作、状态变迁
 - 节点内容展开为 PL6-A 活动节点 ↓
- PL6-A 活动节点 (↑ 处理配置的内容展开, 有任务类型时存在)
 - 任务类型确定后, wft01 在业务语言层捕获/推断主要活动节点 ("部门经理审批, 通过后到财务")
 - wft02 将业务节点转化为引擎 CU 候选 (引擎语言), 补全系统自动节点/异常路径/超时处理
 - wft03 生成活动节点的引擎 CU 配置需求
 - 每个活动节点定义: 处理角色、可执行动作、状态变迁

2.3 PL6-B 正文条目

输出文档的正文条目满足以下三独立条件时, 展开为 PL6-B:

- **独立名称:** 每个条目有独立的名称
- **独立定义要素:** 每个条目有独立的定义要素 (如指标的口径/维度、看板的布局/指标集、风险的概率/影响/缓解措施、问题的根因/纠正措施)
- **独立生命周期:** 每个条目的生命周期独立于容器文档 (可独立创建/修订/废弃)

PL5 输出文档 (正文条目满足三独立)

- 封面实体 (同 §2.2)
- 正文实体 (容器——该清单作为一个整体的元数据)
 - 条目内容展开为 PL6-B 正文条目 ↓
- PL6-B 正文条目 (↑ 正文实体的条目展开, 逐条目)
 - 条目1: 独立名称 + 独立定义要素 + 独立生命周期
 - 条目2: 独立名称 + 独立定义要素 + 独立生命周期
 - ...
 - 条目N

满足三独立: 指标清单 (指标条目)、看板清单 (看板条目)、计划清单 (计划条目)、风险清单 (风险条目)、问题清单 (问题条目)、行动项清单、改进措施清单等——PCA 场景的全部输出文档正文条目均满足三独立。

不满足三独立: 事务类文档的物料行 (运行时交易数据, 无独立生命周期)、内容类文档的议题条目 (内容实例, 无独立定义要素)。轻量文档无封面, 正文即是全部内容, 不展开 PL6-B。轻量文档同样可配置任务类型 (如会议纪要配置流程进行审批流转), 任务状态由对应引擎在正文实体上直接管理。

2.4 轻量文档的组成

轻量文档（PL5）	
├	正文实体（无封面行，1 行 = 1 文档内容条目实例）
├	└ 被功能引擎的表单能力直接转化为台账（列表/树表）
├	└ 无封面，正文即文档
├	处理配置（可选的任务类型：流程/工单/计划）
├	└ 任务状态由对应引擎在正文实体上直接管理，无封面行承载
└	支持信息（→其他文档的多对多引用）
	└ 被引用目标为轻量文档时，直接指向正文实体行

2.5 设计规则

规则	说明
R1	正文实体始终存在——所有文档都有正文（1 行 = 1 文档内容条目实例）
R2	封面实体按需——仅当文档有独立于正文 CRUD 的生命周期时才存在
R3	封面实体存在时，DocumentCover 行与正文行严格 1:1 + <code>derived_from</code> 追溯链（每版本独立，旧版本归档）
R4	场景业务中第一个有任务类型的输出文档，其任务类型必然是计划。此文档本体是开发/记录类文档，正文为产品数据或管理数据，非"计划清单"
R5	任务间可任意触发——任何任务类型（计划/流程/工单）均可触发任何任务类型（包括同类型），触发关系由业务需要决定，无类型限制
R6	有任务类型时，必须展开 PL6-A 活动节点——wft01 在业务语言层捕获/推断主要节点，wft02 转化为引擎 CU 候选并补全系统节点/异常路径，wft03 生成 CU 配置。正式文档的活动节点附着于封面行，轻量文档的活动节点附着于正文实体
R7	正文条目满足三独立条件（独立名称 + 独立定义要素 + 独立生命周期）的，展开为 PL6-B。事务类文档的物料行和内容类文档的议题条目不满足三独立，不展开 PL6-B

2.6 任务间触发关系

任务类型三选一：正式文档和轻量文档均可配置零个或一个任务类型——**计划**（策划→发布）、**流程**（审批流转）、**工单**（派发/处理/验收）。区别在于：正式文档的任务状态由封面行承载（处理配置附着于 DocumentCover 行），轻量文档的任务状态由对应引擎在正文实体上直接管理（无封面行）。

任务关系规则：任务间触发规则见 §2.5 R5——任何任务类型（计划/流程/工单）均可触发任何任务类型（包括同类型），触发关系由业务需要决定，无类型限制。

2.7 PCA 场景的 P/C/A

PCA 场景业务与 D 类完全同构——由输出文档序列构成，每个输出文档按需配置处理配置（任务类型）。除遵循 §2.5 R4（首个有任务类型的文档为计划，本体为开发/记录类文档）外，PCA 场景还必须包含一个计划清单输出文档（见下文），两者是不同的文档实例。

PCA 场景**必须包含一个输出文档为计划清单**（正文条目满足三独立，展开 PL6-B）。其正文条目中：

- 第 1 条为"策划本次计划清单及其余清单"，任务类型为**计划**（策划→发布）。策划主管按周期进入 PCA 菜单，创建该条目后即按计划引擎的生命周期运行

- **后续条目**逐条对应 P 阶段各输出文档的策划任务——"制定指标清单"、"识别风险清单"、"设计看板清单"、"梳理问题清单"等

第 1 条发布完成后，触发后续条目的执行。后续条目逐一触发各自对应的输出文档——这些文档的任务类型**按业务需要选择**（流程/工单/计划），不强制为计划。

C（监控）和 A（改进）是任务链运行时推进的结果：P 阶段触发的计划/流程/工单执行 → 指标引擎持续度量 → 看板呈现 → 发现偏差 → 生成风险条目/问题条目（→风险清单/问题清单，条目按需走流程或工单处理） → A 阶段改进完成。改进完成后可触发下一轮 P 的计划清单第 1 条（新一轮策划），也可由策划主管按周期手动启动。

三、基于引擎生成业务

3.1 业务及层级

EOS 平台使用四套视角层级描述业务在不同维度上的组织结构：

视角	编号	自上而下
ML（菜单视角）	M0→M4	平台 → 一级菜单 → 二级菜单 → 三级菜单 → 末级菜单（→SL5）
SL（角色场景视角）	SL0→SL6	平台 → 业务域 → 产品类型 → 角色 → 角色职责 → 场景业务 → 功能表单/主子表单
PL（输出产品视角）	PL0→PL6	平台 → 业务域 → 产品类型 → 构件 → WBS → 输出文档 → 活动节点/正文条目
EL（引擎视角）	EL1→EL3	菜单引擎 → 功能引擎 → 实体引擎

其余视角层级的完整定义见 23-eos-output-architecture.md 和 24-eos-business-assets.md。

PL5 输出文档的组成见 §二——封面实体 + 正文实体（1:1）+ 支持信息引用 + 处理配置。

PL6 活动节点/正文条目——PL5 之下设 PL6，分为两个子类型：

- **PL6-A 活动节点**：有任务类型时，任务的活动节点序列构成 PL6-A——wft01 在业务语言层捕获/推断主要节点，wft02 转化为引擎 CU 候选并补全系统节点/异常路径，wft03 生成 CU 配置。正式文档的活动节点附着于封面行，轻量文档的活动节点附着于正文实体。跨三个设计阶段逐级展开，是架构节点而非运行时配置
- **PL6-B 正文条目**：正文条目满足三独立条件（独立名称 + 独立定义要素 + 独立生命周期）的，展开为 PL6-B。PCA 场景的全部输出文档（指标清单/看板清单/计划清单/风险清单/问题清单/行动项清单/改进措施清单等）正文条目均满足三独立。事务类文档的物料行（运行时交易数据）和内容类文档的议题条目（内容实例）不满足三独立，不展开 PL6-B

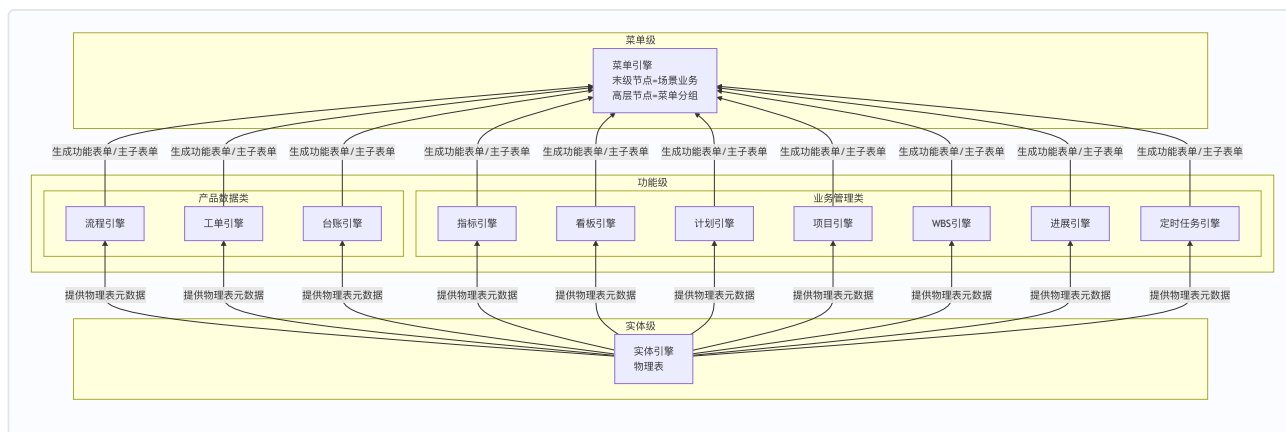
"输出文档"的指代：本文档及 biz 路径各 Skill 中提及的"输出文档"，均指代文档的**内容本身**而非其容器或文件名。例如"相关方需求文档"指需求条目本身，"采购履约看板"指该看板的定义与布局本身。看板上的每个具体看板/指标为该输出文档的正文内容条目，在 PL6-B 层展开为独立设计对象。文档的承载表单（如 @engine-kanban）仅为组织容器，不影响每个条目作为独立 PL6 节点的地位。

输出文档按生成方式分为两类：

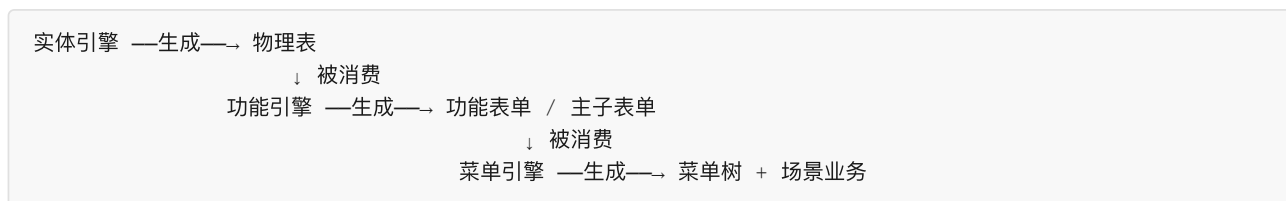
- **内置 CAX（平台内引擎生成）**：内容由 EOS 功能引擎直接生成，沿引擎间制品流转链逐级生成——对应物理表（实体引擎）→转换为功能表单→按主子关系组装为主子表单。
- **外置 CAX（平台外工具生成）**：文档以文件名及附件形式回传至 EOS 平台，平台负责管理其任务、状态、责任人、版本、附件、评审和归档。

3.2 引擎及层级

EOS 平台通过三层引擎逐级生成：下层引擎生成制品，上层引擎消费该制品后生成自身制品，自底向上逐级递进，从物理表一直生成到用户可见的菜单：



简化版引擎间制品流转：



各引擎为生成器：下级引擎生成制品，上级引擎消费该制品后生成自身制品，逐级递进直至平台可发布的末级菜单。

引擎全景：

序号	引擎ID	名称	层级	一句话定义
1	@engine-entity	实体引擎	实体级	基于数据库生成物理表，提供增删改查、导入/导出和追踪关系 CRUD 等基本功能
2	@engine-flow	流程引擎	功能级（产品数据类）	配置流程节点、流转规则、审批行为和状态迁移
3	@engine-workorder	工单引擎	功能级（产品数据类）	配置工单类型、派发规则、处理流程、验收标准和归档策略
4	@engine-ledger	台账引擎	功能级（产品数据类）	基于物理表生成台账功能表单并按主子关系装配为主子功能表单，用于台账类业务（设备台账等）
5	@engine-project	项目引擎	功能级（业务管理类）	基于物理表配置项目对象、阶段、里程碑、成员和项目状态
6	@engine-wbs	WBS 引擎	功能级（业务管理类）	基于物理表配置 WBS 层级、分解规则、节点属性和进度汇总方式
7	@engine-task	计划引擎	功能级（业务管理类）	基于物理表配置计划任务、时间约束、依赖关系和执行状态
8	@engine-kanban	看板引擎	功能级（业务管理类）	配置指标维度、度量口径、聚合规则、展示方式和看板视图布局，为物理表生成指标型+看板型功能表单
9	@engine-indicator	指标引擎	功能级（业务管理类）	定义指标的名称/口径/目标/维度/图示，生成指标结果功能表单（列表模式/图示模式 2×3网格）
10	@engine-progress	进展引擎	功能级（业务管理类）	基于物理表配置进展填报、进度口径、状态更新和进展记录
11	@engine-scheduler	定时任务引擎	功能级（业务管理类）	基于物理表配置定时触发规则、任务事件和执行策略
12	@engine-menu	菜单引擎	菜单级	配置菜单树节点和场景编排

EL 三层中，「功能引擎」是层级（EL2）而非独立引擎资产。上表中 10 个具体功能引擎均归属功能级。菜单引擎（EL1）和实体引擎（EL3）同时是层级名称和该层级唯一的引擎资产。

引擎职责：

- **实体引擎**：基于数据库生成物理表，提供增删改查、导入/导出和追踪关系 CRUD 等基本功能。物理表是上层引擎的元数据基础。
- **功能引擎**：将物理表转换为功能表单并按主子关系（1+N）组装为主子表单，配置行/列级数据访问权限。承载四方面能力：① 表单能力（结构/字段/布局/校验/行为/发布）；② 功能能力（各具体引擎为表单增加的功能）；③ 组装能力（主子表单组装、子表单显示规则匹配、双击下钻）；④ 权限能力（数据行/列对角色/人员的可见、可编辑或不可访问约束）。
 - **产品数据类**（流程/工单/台账）：支撑产品数据文档的开发与管理——流程引擎管理审批流转，工单引擎管理派发/处理/验收/关闭，台账引擎将物理表转换为台账功能表单并按主子关系装配为主子功能表单（用于设备台账、风险清单、问题清单、行动项清单、改进措施清单等台账类业务）。封面-正文 1:1 绑定由功能引擎的组装能力统一完成，不设独立文档引擎。

- **业务管理类**（指标/看板/项目/WBS/计划/进展/定时任务）：支撑过程管理数据的生成与管理——指标引擎管理指标定义与双模式展示（列表/图示 2×3 网格），看板引擎管理看板布局与视图泳道

- **菜单引擎**：生成并维护多级菜单树，末级节点编排场景业务。

3.3 配置单元

配置单元选配模型：每个引擎包含若干配置单元（可独立选配），部分配置单元即可让业务运行——引擎提供能力池，配置人员按业务需要从中挑选组合。

配置单元的功能本质：CU（配置单元）是在实体页面（§1.2 的六种页面类型之一）上增加一组业务操作能力。业务配置人员打开 CU 对应的功能表单，操作页面上的功能按钮、编辑各配置要素，提交后系统校验信息的正确性和完备性，通过后保存配置信息。引擎按 CU 定义生成运行制品（物理表、功能表单、菜单入口），配置人员或业务使用人员即可使用该制品及其功能。

具体而言：

- **实体引擎 CU** → 定义实体及字段，在页面类型①/②/④/⑥上提供增删改查能力。封面实体（DocumentCover 表）和正文实体均由此类 CU 定义
- **功能引擎 CU**（如 @engine-flow CU）→ 在已有实体页面上增加审批流转按钮和流程节点配置能力
- **功能引擎 CU**（如 @engine-workorder CU）→ 在工单页面上增加派发/处理/验收按钮和生命周期管理能力
- **功能引擎 CU**（如 @engine-ledger CU）→ 在台账页面上将物理表转换为台账功能表单（列表/树表），按主子关系装配为主子功能表单，提供条目化数据的增删改查和导入/导出能力
- **功能引擎 CU**（如 @engine-indicator CU）→ 在指标页面上增加指标定义、口径配置、列表/图示双模式展示切换和 2×3 网格布局配置能力
- **功能引擎 CU**（如 @engine-kanban CU）→ 在看板页面上增加看板布局、视图泳道和展示方式配置能力

3.4 业务组装

从输出文档到引擎支撑（自顶向下）：STR-F 架构末级节点是场景业务，场景业务由输出文档序列细化定义。从输出文档出发，按生成方式分两条路径确定支撑引擎链：

- **内置 CAX**：内容由平台内引擎生成。正式文档——封面实体和正文实体由实体引擎生成物理表，封面由功能引擎的表单能力转化为封面表单（单据头样式），正文由对应引擎（流程/工单/指标/看板/项目/WBS/计划/进展/台账等）转化为正文功能表单；封面表单与正文表单由功能引擎的组装能力按 1:1 主子关系组装。轻量文档——正文实体由实体引擎生成物理表，正文由对应引擎转化为功能表单，不产生封面表单和主子组装。若正文行与其他文档存在层级引用关系（如双击下钻），按 1+N 扩展组装
- **外置 CAX**：内容在平台外工具生成，以文件名及附件回传。输出文档挂接到计划功能表单，由计划引擎（@engine-task）管理任务、状态、责任人、版本、附件、评审和归档；附件能力由实体引擎的内置子业务配置单元提供；组装时 N=0

无论内置或外置 CAX，全部主子表单序列最终由菜单引擎末级节点编排为场景业务，并生成平台菜单入口。

组装工序：

物理表生成：实体引擎为输出文档的封面实体（如需要）和正文实体生成物理表，提供增删改查、导入/导出和追踪关系 CRUD 等基本数据操作能力。轻量文档仅生成正文实体物理表。

1:1 主子组装：正式文档的封面实体物理表由功能引擎的表单能力转化为封面表单（单据头样式），正文实体物理表由对应引擎转化为正文功能表单；功能引擎的组装能力将二者按 1:1 主子关系组装为一个主子表单。轻量文档无封面，正文功能表单即文档本身，此步跳过。

1+N 下钻扩展：若正文行与其他文档存在层级引用关系（双击下钻到目标文档），在 1:1 基础上扩展——目标文档的功能表单作为子表单挂接到当前主子表单。

菜单编排：全部主子表单序列（含内置 CAX 和外置 CAX 的产出）由菜单引擎末级节点编排为场景业务，按输出文档的流转顺序排列为顶部 TAB，生成平台菜单入口。外置 CAX 文档不产生功能表单（N=0），在菜单中展示为计划引擎管理的任务与附件入口。

四、操作页面组件与构件

本章定义 EOS 平台为用户提供的操作页面组件体系、构件分类，及其与引擎配置单元（CU）的对应关系。操作页面组件是平台 UI 的结构层次，构件是安装在组件上的可复用功能单元。与 §1.2 的功能表单页面类型（描述单个功能表单的布局模式）正交——页面类型是组件的组装模式，组件是构成页面类型的积木。

4.1 操作页面组件

操作页面组件按容器-内容层级组织：

多级横版（根容器）

- ├ 菜单栏（左侧导航）
- ├ 窗口区
 - ├ 左窗 — TAB 栏 — [列表页面] [单据页] [指标页面]
 - └ 右窗（分屏时出现）- TAB 栏 — [列表页面] [单据页] [指标页面]
- ├ 辅助栏（右侧可展开）
- └ 面包屑导航（底部）

弹窗页为模态浮层，不参与窗口层级。

序号	组件	层级角色	描述
1	多级横版	根容器	全窗口布局框架。初始态=菜单栏+单窗+辅助栏；分屏态（双击下钻触发）=菜单栏+左右双窗+辅助栏。窗口顶部 TAB 对应当前场景业务的功能表单序列
2	列表页面	内容组件	在窗口 TAB 内展示，数据以单级列表或树形表格呈现。承载数据浏览、筛选、排序、行选择和批量操作功能
3	单据页	内容组件	以单据布局（字段名
4	辅助栏	辅助面板	右侧可展开信息面板，以两列单据布局显示当前选中行的关键字段。可在任意窗口上独立展开/关闭，行为见 §1.3
5	弹窗页	模态浮层	临时性数据选择或编辑窗口，模态覆盖在当前窗口之上。典型场景：字段值引用其他实体时弹窗选择
6	指标页面	内容组件	在窗口 TAB 内展示，以图示（柱/折线/饼/仪表盘等）呈现指标数据，支持 2×3 网格布局

与 §1.2 页面类型的正交关系：§1.2 的六种页面类型是本表组件的**组装模式**——页面类型决定了用哪些组件、如何组合：

页面类型	组件组合
① 单窗列表+辅助栏	多级横版(有菜单栏,有TAB栏,单窗) + 列表页面 + 辅助栏
② 单窗树表+辅助栏	多级横版(有菜单栏,有TAB栏,单窗) + 列表页面(树表) + 辅助栏
③ 单窗单据页	多级横版(有菜单栏,有TAB栏,单窗) + 单据页 + 辅助栏(可选)
④ 左窗主表+右窗子表+辅助栏	多级横版(有菜单栏,有TAB栏,双窗) + 列表页面(左窗) + (列表页面 单据页)(右窗) + 辅助栏
⑤ 全屏单据页	多级横版(无菜单栏,无TAB栏) + 单据页 + 辅助栏(可选)
⑥ 弹窗	弹窗页(模态) + (列表页面 单据页)

同一页面组件可出现在不同页面类型中，页面类型决定了组件的组合方式和布局——两者正交。

4.2 构件

构件是安装在操作页面组件上的可复用功能单元。同一构件在不同页面组件上提供语义相同、形态适配的功能。构件由引擎按 CU 定义生成并渲染到目标页面组件上。

构件按功能分为四类：

一、展示类（决定数据以何种视觉形态呈现）

构件	适用页面组件	说明
列定义	列表页面	列的字段绑定、宽度、对齐、格式化、是否可排序/可筛选
字段布局	单据页、辅助栏	字段分组、排列（单列/双列/多列）、标签位置、是否只读
图表类型	指标页面	柱状图/折线图/饼图/仪表盘/雷达图/数字卡片，含坐标轴和图例配置
卡片	列表页面、指标页面	看板卡片布局——标题/摘要/状态标签/负责人头像，用于看板视图
泳道	列表页面、指标页面	按维度（状态/负责人/阶段）水平或垂直分组的卡片/条目容器
甘特条	列表页面	时间轴上的条形图——起止日期/进度/依赖线，用于计划视图
树节点	列表页面	树形结构的展开/折叠节点——层级缩进/节点图标/父子连线，用于 WBS、BOM、组织树
阶段/里程碑	列表页面、单据页	项目阶段和里程碑的菱形/旗标记——计划日期/实际日期/完成状态

二、交互类（决定用户如何操作数据）

构件	适用页面组件	说明
功能按钮	全部	工具栏按钮/操作区按钮/行内按钮——触发引擎动作（提交/审批/驳回/派发/验收/导出等）
右键菜单项	列表页面、指标页面	选中行后的上下文操作菜单——与功能按钮等效，提供快捷操作入口
快捷键绑定	全部	键盘组合键→引擎动作映射——如 Ctrl+S 保存、Ctrl+Enter 提交
下钻	列表页面、指标页面	双击数据行→触发分屏或新窗口，打开关联文档的功能表单
行选中	列表页面	单选/多选模式——选中后触发辅助栏更新、右键菜单可用项切换、批量操作按钮启用
TAB 切换	多级横版	窗口顶部 TAB 点击→切换当前显示的功能表单；面包屑导航双击→跳转至历史层级

三、数据类（决定数据如何被筛选、排序和导入导出）

构件	适用页面组件	说明
筛选器	列表页面	列头筛选/高级筛选面板——字段条件组合筛选，支持保存为个人视图
排序器	列表页面	单列点击排序/多列组合排序——升序/降序，支持自定义排序规则
分页器	列表页面	数据分页加载——每页条数/页码跳转/总条数显示
导入	列表页面	文件上传→解析→校验→批量写入，含导入模板下载和错误行报告
导出	列表页面、指标页面	当前视图数据导出为 Excel/PDF——含筛选条件和列选择

四、反馈类（决定系统如何向用户反馈状态和结果）

构件	适用页面组件	说明
通知提示	全部	操作结果 Toast（成功/失败/警告）——短暂浮出后自动消失
进度条	单据页、弹窗页	长耗时操作的进度展示——百分比/剩余时间/取消按钮
状态标签	全部	数据状态的可视化标签——审批中(黄)/已通过(绿)/已驳回(红)/草稿(灰)
校验提示	单据页、弹窗页	字段级/表单级校验错误提示——红色边框+错误文本，阻止提交

4.3 CU 与操作页面组件及构件的对应关系

CU 是构件到页面组件的生成指令。引擎读取 CU 定义，在目标页面组件上渲染指定构件。同一 CU 可跨页面组件生成不同形态的构件——如审批按钮在列表页面渲染为工具栏按钮，在单据页渲染为底部操作区按钮。

配置时：CU 对应一个硬代码实现的配置页面（非功能引擎生成）。配置人员在配置页面上编辑 CU 的配置要素，提交后引擎校验并保存。

运行时：引擎按 CU 定义生成运行期制品——在指定的操作页面组件上渲染构件及其交互行为。

端到端示例——@engine-flow CU：

配置时：
 配置人员打开流程配置页面（单据页）
 → 编辑流程节点、流转规则、审批角色、状态迁移
 → 提交 → 引擎校验并保存 CU 定义

运行时：
 引擎读取 CU 定义
 → 在正文单据页上渲染 [提交审批] 按钮（交互类·功能按钮构件）
 → 在正文单据页上渲染 流程节点列表（展示类·列定义构件）
 → 在正文单据页上渲染 当前节点状态（反馈类·状态标签构件）
 → 审批人打开单据页 → 渲染 [通过]/[驳回] 按钮 → 点击后引擎执行状态迁移

4.4 扩展规则

业务需求驱动操作页面组件及构件的扩展，分三级判定：

1. 构件级：现有页面组件可承载，仅需新增或修订构件。

触发场景	示例
新增操作方式	列表页面增加批量导入按钮构件（交互类）
新增展示形态	指标页面增加雷达图图表类型构件（展示类）
新增数据操作	列表页面增加自定义排序规则构件（数据类）
新增反馈方式	单据页增加保存前差异对比提示构件（反馈类）

→ 现有 CU 扩展属性即覆盖，或新增独立 CU。

2. 页面组件级：现有页面组件无法承载新交互模式，需新增组件类型。

判定标准：新需求是否引入了现有六种组件无法表达的**容器-内容关系或交互模式**。

示例：若业务需要"多级横版内嵌一个可拖拽调整大小的分隔面板"，超越了多级横版固定比例分屏的交互模式 → 需评估是否新增页面组件类型。

→ 新增页面组件类型须配套新增 CU（见下）。

3. CU 级：新构件或新页面组件需要 CU 支撑其配置→生成→渲染链路。

→ 形成 FR-ENG 线索，经 eng 路径（§八）推进——wft01-eng 判定归属引擎 → wft02-eng 设计 CU → wft03-eng 设计操作页面与构件 → wft05-eng 展开前后端组件。

biz 路径在设计中发现构件或页面组件缺口时，输出 FR-ENG 线索并标注缺口层级（构件 / 页面组件 / CU）和触发场景。

五、三类产品 A1/A2/Bn

EOS 平台的业务配置和运行围绕三类产品展开：

产品	归属域	构件	末级节点
A1	流程与IT	引擎	CU（配置单元）
A2	各业务域	角色	场景业务
Bn	各业务域	产品分解类型（自研件/外购件/复用件/项目等）	构件的产品数据

三类产品的完整架构定义见 [23 号资产](#)。

5.1 A1产品

A1 是平台引擎产品，归属流程与IT域。构件为引擎（@engine-entity、@engine-flow 等 12 个），末级节点为 CU（配置单元，详见 [25 号资产 §2](#)）。

每个 CU 定义一个可独立配置、校验、发布的引擎元数据对象——配置人员在 CU 配置页面上编辑配置要素，提交后引擎校验并保存，运行时按 CU 定义生成运行期制品（物理表/功能表单/菜单入口+构件）。CU 的完整定义见 §3.3 和 §4.3。

对应设计路径为 **eng 路径**：wft01-eng（场景业务）→ wft02-eng（配置单元）→ wft03-eng（操作页面）→ wft05-eng（前后端组件）。

STR-E 的双重身份：wft01-eng 产出的 STR-E 节点（引擎场景业务）在 A2 视角下是 PL5 末级节点——即流程与IT域中**业务配置人员**的场景业务（§5.2）；在 A1 视角下，该场景业务对应的引擎是 A1 的 PL3 构件节点。一个引擎对应一个 STR-E（~12 引擎 → ~12 STR-E），引擎的配置业务（从打开配置页到运行期能力可用）即为该 STR-E 的端到端闭环。CU 是 STR-E 的细分节点——与 STR-F 下辖输出文档的层级关系完全对称。

5.2 A2产品

A2 是各业务域的角色场景，构件为角色（如采购员、库存管理员），末级节点为场景业务。沿角色场景视角（SL）展开：SL3 角色类型 → SL4 角色职责 → SL5 场景业务 → SL6 功能表单/主子表单。

每个 A2 产品回答：某角色在哪个菜单入口、按什么顺序使用哪些功能表单/主子表单、完成端到端的职责闭环。场景业务（SL5）与 STR-F 末级节点 1:1 映射。

对应设计路径为 **biz 路径的前半段**：wft01-biz（场景业务）→ wft02-biz（输出文档，组装 A2 产品业务架构）。

流程与IT域的 A2 产品：流程与IT域同样有 A2 产品——**业务配置人员**是该域的核心角色类型（此外还有业务架构管理员、数据治理员，见 §六）。业务配置人员的场景业务（SL5）即 STR-E ——一个引擎的配置业务为一个场景业务（~12 引擎 → ~12 STR-E）。这些 STR-E 与采购/生产/售后等业务域的 STR-F 在架构层级上完全同构：同样是 SL5 末级节点、同样下辖细分节点（CU vs 输出文档）、同样是角色端到端职责闭环。区别在于场景业务的产出物不是业务文档，而是**可用的平台能力**。

5.3 Bn产品

Bn 是各业务域的输出产品，沿产出视角（PL）展开：PL1 业务域 → PL2 E2E 项目业务 → PL3 构件类型 → PL4 WBS → PL5 业务输出文档。每个 Bn 产品回答：某类输出文档属于哪个业务域/构件类型，需要哪些前序文档和业务

规则才能生成。

构件类型是产品的分解分类节点，取决于产品域的性质。不同构件类型产生不同的输出文档集合——自研件需要需求/设计/实现/测试全套文档，外购件需要采购规格/验收标准/合格证，复用件仅需适配说明和集成测试报告。

产品域	构件类型示例
设备产品（如发电设备）	系统、系统电缆（自研）、机箱及电连接（自研）、电子模块（自研）、电路板（自研）、软件模块（自研）、结构模块（自研）、光学组件（自研）、复用构件、外购构件、项目
业务变革产品（如企业运营体系建设）	业务体系、E2E 业务（自研）、E2E 业务（咨询）、基础数据、部门、项目

后缀（自研/咨询/外购/复用）标注构件的开发方式。项目构件是特殊的构件类型，承载项目管理类文档（计划清单/风险清单/问题清单/行动项清单），不产出实物产品数据。

两类产品数据：每类构件下辖的输出文档承载两种产品数据——

类型	说明	示例
交付类	交付给客户/用户的实物产品、技术方案及过程记录，沿端到端业务链逐级生成和流转	需求与方案阶段——相关方需求文档/运行概念/系统需求规格/产品架构定义；设计与实现阶段——需求规格/技术方案/实现方案/测试报告；制造与交付阶段——设计图纸/物料清单/工艺文件/出厂检验报告/安装验收报告
管理类	支撑业务运转的策划、监控和改进数据	指标/看板/计划清单/风险清单/问题清单/行动项清单/改进措施清单

交付类数据是企业运营的主价值链——前一文档的正文数据成为后一文档的输入，从客户需求一直贯通到交付验收。管理类数据是 PCA 闭环的产物，监控主价值链的运行状态并驱动改进。EOS 平台将两类产品数据均承载为输出文档，由对应引擎转化为功能表单，使业务人员在平台上直接创建、编辑、流转和追溯。

与 A2 的关系：A2 和 Bn 是对同一业务的两种互补视角——A2 描述**角色怎么工作**（谁→在哪个菜单→按什么顺序→使用哪些功能表单→完成职责闭环），Bn 描述**角色产出了什么**（什么产品数据→属于哪个域→经过哪些前序文档→流转到哪里）。两者通过功能表单/主子表单交汇：同一张功能表单，在 A2 侧是角色的操作界面，在 Bn 侧是产品数据的载体。

对应设计路径为 **biz 路径的后半段**：wft02-biz 装配 Bn 产品业务架构 → wft03-biz 生成各层配置需求。

六、平台资产治理业务

平台资产治理是 EOS 平台的**内置场景业务**（A2 产品）——平台自身的管理对象（菜单、架构资产、数据标准、权限）就是平台的业务数据，平台提供对应的引擎和功能表单直接管理这些对象，治理人员使用平台能力治理平台。各治理领域与平台引擎的对应关系见下表。STR-F 已预置在 02-*.md 中。

治理操作遵循统一的生命周期模式：**定义/登记** → **审核/发布** → **监控/审计** → **变更/修订** → **废弃/归档**。

治理领域	执行角色	管理对象	平台支撑
菜单配置	业务架构管理员	菜单树节点（M0→M4）	菜单引擎提供菜单树维护功能表单
资产管理	业务架构管理员	23~28 号架构资产	实体引擎+台账引擎提供资产登记/变更/查询功能表单
数据治理	数据治理员	数据字典/编码规则/状态库/质量规则	实体引擎+台账引擎提供标准维护功能表单
权限治理	业务架构管理员	角色/菜单权限/数据权限	菜单引擎+功能引擎提供权限配置功能表单

6.1 菜单配置

菜单配置是平台导航结构的管理业务——EOS 平台的菜单树本身就是平台的管理对象，菜单引擎提供菜单树维护功能表单，业务架构管理员直接在上面创建、修改、删除菜单节点。

管理对象：菜单树节点，分为两类——

- **非末级节点（M1→M3）：**承载分组和导航，定义层级归属、显示顺序、可见角色/人员
- **末级节点（M4）：**定义所包含的功能表单/主子表单序列（即场景业务）、显示顺序和依赖关系，以及可见角色/人员

核心操作：

操作	说明
新建节点	确定层级归属（M1~M4）、显示顺序，配置可见角色/人员；末级节点需指定场景业务映射
修改节点	调整名称/层级/顺序/可见性/场景业务映射；记录变更原因和影响范围
删除节点	检查是否存在子节点或已分配的场景业务→标记为废弃→确认无引用后物理删除
可见性配置	按角色或人员配置节点的可见/不可见——可见性由菜单引擎在运行时执行
发布	菜单变更经审核后发布，菜单引擎重新生成菜单树并生效

菜单变更操作自动生成变更记录（变更内容/操作人/时间/影响范围），形成菜单配置的审计追溯链。

6.2 资产管理

资产管理是平台架构资产的维护业务——23~28 号资产是平台的管理对象，实体引擎生成资产物理表，台账引擎将其转化为台账功能表单，业务架构管理员直接在平台上登记、变更、查询和废弃资产。

管理对象：23~28 号架构资产——

资产编号	资产名称	内容
23	产品架构资产	产品架构节点定义、PL/SL/ML/EL 层级归属、节点间关系
24	业务资产	业务域/业务类型/端到端业务/角色/WBS 定义
25	引擎资产	引擎定义、CU 定义、配置要素规格
26	文档定义资产	输出文档封面/正文结构、字段定义、转化引擎映射
27	资源资产	数据字典/编码规则/状态库/组织结构的平台资源配置
28	看板指标资产	指标定义（口径/维度/目标）、看板布局定义

核心操作：

操作	说明
登记	新资产创建后登记入库，分配唯一资产 ID，建立初始引用关系
变更	资产属性或关系变更→记录变更内容/原因/影响范围→更新引用关系→触发受影响资产的级联检查
废弃	资产不再使用→标记为废弃→解除引用关系→归档，保留变更历史
引用关系维护	维护资产间的引用和追溯关系——如 23 节点→24 业务资产→25 引擎资产 的引用链
一致性检查	定期或变更触发时检查资产间引用完整性——是否存在悬空引用/版本不一致/双向引用缺失

资产变更的级联影响：当某一资产发生变更时，系统检查所有引用该资产的下游资产——若下游资产依赖的字段/关系/定义发生变化，标记为"待更新"并通知资产管理。

资产变更操作自动生成变更记录，一致性检查可手动或定时触发，生成检查报告。

6.3 数据治理

数据治理是平台数据标准的维护业务——数据字典、编码规则、状态库、质量规则本身是平台的管理对象，平台提供标准维护功能表单，数据治理员直接在上面定义和修订标准。各引擎在运行时读取这些标准以保持数据一致性。

管理对象：

对象	说明	示例
数据字典	平台级字段定义标准——字段名称/数据类型/长度/格式/默认值/值域	"采购申请单号"字段统一为 PO-yyyyMMdd-XXX
编码规则	实体/文档/资产的编码生成规则——前缀/日期/序列/校验位	物料编码：MAT-品类码-流水号
状态库	平台级状态定义——状态名称/状态值/适用引擎/状态迁移规则	"审批中/已通过/已驳回/已撤回"统一状态集
组织/人员主数据	部门/岗位/人员的基础数据，供权限分配和流程审批引用	组织树/岗位定义/人员归属
质量规则	数据校验规则——唯一性/必填性/格式/引用完整性/业务规则	"采购申请单金额>0""物料编码不可重复"

核心操作：

操作	说明
标准定义	新增数据字典条目/编码规则/状态定义→审核→发布
标准变更	已发布标准的修订→评估对已有引擎配置和业务数据的影响→变更→通知受影响方
质量监控	配置数据质量规则→引擎按规则校验运行期数据→发现违规生成数据问题单
问题处理	数据问题单→分析根因→修正数据或修订标准→关闭

标准变更操作自动生成变更记录，质量规则校验结果生成数据问题单（由工单引擎驱动处理）。

6.4 权限治理

权限治理是平台访问控制的管理业务——角色、菜单权限、数据权限本身是平台的管理对象，菜单引擎和功能引擎提供权限配置功能表单，业务架构管理员直接在上面分配和维护权限。

管理对象：

权限类型	承载引擎	内容
菜单权限	菜单引擎	菜单节点对角色/人员的可见或不可见——控制能进入哪些场景业务
数据权限	功能引擎	物理表数据的行/列对角色/人员的可见/可编辑/不可访问——控制能看到和操作哪些数据

核心概念：

- **角色**：权限分配的基本单位。角色按业务职责定义（如采购员、库存管理员、部门经理），人员分配到角色，角色分配到权限
- **角色层级**：上级角色继承下级角色的权限——如部门经理继承部门内所有角色的菜单权限
- **权限分离**：互斥角色不能同时分配给同一人员——如采购申请人和采购审批人

核心操作：

操作	说明
角色定义	创建/修改/废弃角色——定义角色名称/职责范围/角色层级
菜单权限分配	配置角色可见的菜单节点→菜单引擎运行时按角色过滤菜单树
数据权限分配	配置角色对物理表数据的行/列访问规则→功能引擎运行时执行过滤
人员分配	人员分配到角色→人员调动时重新分配→离职时回收全部角色
权限审计	定期审计权限分配是否合理——是否存在权限过大/权限分离冲突/僵尸权限

权限变更操作自动生成分配记录，权限审计可手动或定时触发，生成审计报告。

治理 STR-F 的完整清单以 02-*.md 为准，AI 运行时动态读取。上述四类治理业务当前 OR 承接为空，由人类在 EOS 平台建设过程中逐批增补。

七、业务系统之系统设计

biz 路径将业务原始需求（FR-BIZ OR）逐级转化为可交付构件开发的业务配置需求方案。三个 Skill 形成设计接力链——每个 Skill 的输出是下一个的输入，语言从业务方听得懂的**业务语言**逐步过渡到构件开发可消费的平台语言：

FR-BIZ OR	→	wft01-biz	→	wft02-biz	→	wft03-biz
		场景业务方案		业务架构方案		业务配置需求方案
		业务语言		架构语言		平台语言
		"我听懂了"		"方案是对的"		"可以开始开发了"

Skill	输入	产出	末级节点	核心命题
wft01-biz	FR-BIZ OR 条目	场景业务方案——场景业务 + PL5 输出文档序列 + PL6 子节点/条目初步判定	场景业务	向业务方证明"我听懂了你的业务"
wft02-biz	STR-F 节点	业务架构方案——A2（角色→场景业务→功能表单/主子表单）+ Bn（构件类型→WBS→输出文档 + PL6-A 节点需求定义 + PL6-B 条目需求定义）	输出文档（PL5）， 细分 PL6-A / PL6-B	设计正确的方案——业务语言→架构语言
wft03-biz	BA 节点	业务配置需求方案——功能表单需求定义 + 操作页面需求定义 + 引擎能力缺口线索	功能表单需求 / 操作页面需求	产出可交付构件开发的业务需求规格

三个 Skill 的分工逻辑：

	wft01-biz	wft02-biz	wft03-biz
回答什么问题	要做什么业务	业务应怎么做	业务要怎么配
语言层	业务语言——业务方听得懂	架构语言——桥接业务至平台	平台语言——构件配置需求
产出物	场景业务方案（STR-F）——场景业务 + 输出文档序列 + PL6-A 活动节点与 PL6-B 正文条目初步判定	业务架构方案（BA）——A2 角色场景（功能表单/主子表单）+ Bn 输出产品（输出文档定义 + PL6-A 节点需求定义 + PL6-B 条目需求定义）	业务配置需求方案——功能表单需求定义（字段/布局/交互/展示）+ 操作页面需求定义（页面序列/流转规则/交互需求）+ 引擎能力缺口线索
供谁消费	业务方确认 → wft02-biz	wft03-biz	构件开发流水线 + eng 路径（引擎缺口线索）

相关方需求架构文档 02-*.md 在 wft01-biz 产出 STR-F 后形成。D 和 PCA 场景的处理逻辑完全同构。

与 eng 路径的对称性（详见 §八）：biz 路径的输出文档对应 eng 路径的 CU——都是各自产品的 PL5 末级节点。wft03-biz 的配置需求方案 ↔ wft03-eng 的操作页面，同属 SR-F 层。

7.1 原始需求处理（wft01-biz）

从零散但规范化的 OR 到结构化的场景业务方案。核心使命：**向业务方证明"我听懂了你的业务"**——产出供人类确认理解，而非最终设计方案。

输入 产出
FR-BIZ OR (待处理) → STR-F 场景业务方案 (可以分解分配)
末级节点: 场景业务 (SL5 / PL5)

任务清单:

#	任务	说明
1	推断场景业务节点	判定类型 (执行类 D / 管理类 PCA)、命名、边界定义
2	推断输出文档序列	逐文档识别: 判定封面模式 (正式/轻量)、定义正文实体类型
3	判定任务类型与子节点	业务语言, 供人类确认。判定任务类型 (计划/流程/工单) → 展开 PL6-A 活动节点, 捕获 OR 明示节点。管理类 (PCA) 场景业务需判定 PL6-B 正文条目。R4: 首个有任务类型的文档必然为计划
4	推断任务间关系	业务语言。文档间流转顺序 + OR 明示的任务触发关系
5	基于 E2E 补充业务需求	闭合检查 (D: 从需求提出到需求满足; PCA: 计划→监控→分析→纠正), 检出缺口 → 生成补充 OR 材料
6	推断文档内容支撑引擎	按文档类型匹配引擎 (台账/工单/指标/看板/项目/WBS/计划/进展等), 全量标注 [推断]
7	推断输出文档所属 Bn	推断输出文档所属 Bn 产品及其构件类型

Task 3 中 OR 描述的三种处理情形:

OR 描述情况	处理方式
OR 明确描述活动节点	原样捕获为 PL6-A 子节点
OR 只提任务模式 (如"需要审批")	判定类型, 子节点标 [待wft02推断]
OR 缺失活动节点	推断类型, 子节点标 [待wft02推断]

任务 3 判定 PL5 输出文档的 PL6 子节点类型——有任务类型则展开 PL6-A 活动节点, 正文条目满足三独立则展开 PL6-B 正文条目。部分文档仅有 PL6-A, 部分兼有 PL6-A + PL6-B。wft01 在业务语言层捕获/推断 PL6-A 活动节点, 供人类确认"你理解的任务流转关系是否正确"。

7.2 业务架构开发 (wft02-biz)

将 STR-F 场景业务展开为完整的 BA 方案——A2 (角色怎么工作) + Bn (角色产出什么)。核心使命: **设计正确的方案**, 完成业务语言 → 架构语言的转化。

输入 产出
STR-F 节点 BA 业务架构方案 (可以SR设计)
(可以分解分配 / 待补充分解分配) → 末级节点: 输出文档 (PL5)
细分: PL6-A 活动节点 / PL6-B 正文条目

关键转化——业务语言 → 架构语言：wft01-biz 用业务方听得懂的话描述活动节点和关系，wft02-biz 将其转化为架构语言描述的 BA 设计方案——将任务类型映射到对应引擎，逐 PL6-A 活动节点定义处理角色/可执行动作/状态变迁（全量标注 [推断]），供 wft03-biz 展开为功能表单/主子表单序列和流程类操作页面序列的需求定义：

wft01 输出（业务语言）	wft02 转化（架构语言）
任务类型：计划 / 流程 / 工单	引擎映射：@engine-task / @engine-flow / @engine-workorder
"部门经理审批，通过后到财务"	角色 → 引擎角色配置，动作 → 引擎动作配置，状态变迁 → 引擎状态配置
[待wft02推断] 节点	设计完整节点序列（含系统自动节点/异常路径/超时处理）

任务清单：

#	任务	视角	说明
1	装配 A2 产品业务架构	A2	角色 → 场景业务 → 功能表单/主子表单
2	装配 Bn 产品业务架构	Bn	构件类型 → WBS → 输出文档
3	推断 Bn 构件输出文档	Bn	逐构件类型推断其所辖的输出文档集合
4	基于 Bn 补充业务需求	Bn	构件 WBS 分解中发现的文档缺口 → 生成补充 OR 材料
5	定义输出文档方案	Bn	正式文档：封面字段 + 正文实体结构 + 正文转化引擎 + 支持信息引用；轻量文档：正文实体结构 + 正文转化引擎 + 支持信息引用
6	定义 PL6-A 活动节点需求	Bn	有任务类型时，消费 wft01 的业务节点（业务语言）→ 逐节点定义处理角色、可执行动作、状态变迁、系统自动节点/异常路径/超时处理，全量标注 [推断]。操作页面序列编排由 wft03 负责
7	判定并定义 PL6-B 正文条目需求	Bn	正文条目满足三独立（独立名称 + 独立定义要素 + 独立生命周期）→ 逐条目定义名称、定义要素、生命周期。支撑引擎继承自文档正文支撑引擎（Task 5），无需单独判定。事务类（物料行）和内容类（议题条目）不满足三独立，不展开
8	任务关系引擎化	—	文档间流转顺序 → 输出文档序列编排；任务间触发规则 → 引擎任务间触发规则配置（含触发条件和启动参数）

7.3 业务配置需求开发（wft03-biz）

将 BA 方案展开为功能表单和操作页面的需求定义——描述业务系统需要什么样的功能表单和操作页面。核心使命：**产出可交付构件开发的业务需求规格。**

输入	产出
BA 节点	功能表单需求定义 + 操作页面需求定义
（可以SR设计 / 待补充SR设计）	——（待构件需求定义）
	末级节点：功能表单需求 / 操作页面需求

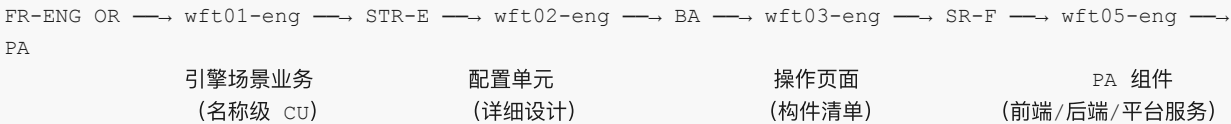
wft03-biz 的产出是业务系统产品架构末级节点的**需求定义**——功能表单和操作页面需要具备哪些字段、布局、交互行为和流转规则。CU 的配置方案和设计方案是构件开发流水线的职责；引擎开发完成后，构件开发流水线按需求定义实现功能表单和操作页面。

任务清单：

#	任务	涉及引擎	说明
1	菜单树方案	@engine-menu	菜单层级结构、末级节点 → 场景业务映射
2	场景业务方案	@engine-menu	功能表单/主子表单序列编排、显示顺序与依赖关系
3	输出文档实体需求	@engine-entity	封面实体字段（DocumentCover 表）+ 正文实体字段（字段名/类型/约束/CRUD），定义文档的数据模型
4	正文功能表单需求	各功能引擎	将正文实体转化为功能表单形态（台账/指标/看板/项目/WBS/计划/流程/工单等），定义字段布局、交互行为和展示方式
5	流程类操作页面序列需求	@engine-flow / @engine-task / @engine-workorder	消费 wft02 的 PL6-A 活动节点需求定义，编排为操作页面序列（审批页 → 派发页 → 验收页等），定义页面间流转规则和每个页面的交互需求
6	任务关系需求	各引擎任务间触发规则	消费 wft02 的任务关系，定义任务间触发规则（触发条件/启动参数）
7	支持信息引用需求	@engine-entity	文档间多对多引用的引用规则及引用角色标签
8	对照引擎能力补充引擎需求	—	逐条需求对照对应引擎的功能范围（91 规范 §3.2 + 25 号资产引擎模型），检出超出或可能超出引擎功能边界的需求 → 生成 FR-ENG 线索（附需求描述和判定依据），转入 eng 路径（§/\）

八、平台能力之系统设计

eng 路径将引擎能力原始需求 (FR-ENG OR) 逐级转化为可交付开发的 PA 组件需求定义 (前端组件 + 后端组件 + 平台服务组件)。与 biz 路径并行但设计对象不同——biz 设计业务场景, eng 设计支撑业务场景的引擎能力。



四个 Skill 形成设计接力链——每个 Skill 的输出是下一个的输入, 从"听懂引擎能力需求"逐步推进到"组件可交付开发":

Skill	核心命题	输入 → 产出	末级节点 (细分)	供谁消费
wft01-eng	向平台方证明"我听懂了你的引擎能力"	FR-ENG OR 条目 → STR-E 引擎场景业务方案	场景业务 (CU 候选, 名称级)	业务配置人员确认 → wft02-eng
wft02-eng	设计可配置的 CU——名称级 → 详细设计	STR-E 节点 → A1 BA 详细设计	配置单元 CU (操作页面: 配置页+业务页)	wft03-eng
wft03-eng	为每个操作页面配齐构件清单	BA 节点 (A1) → SR-F 引擎功能需求	操作页面 (构件)	wft05-eng
wft05-eng	产出可交付开发的 PA 组件需求定义	SR-F 构件清单 + NFR 约束包 → PA 组件需求定义	PA 组件 (前端/后端/平台服务)	构件开发流水线

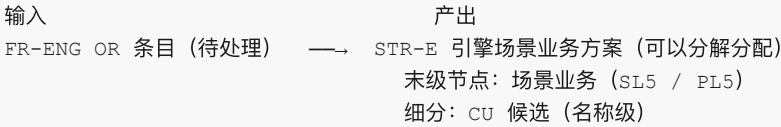
角色: eng 路径的设计角色为流程与IT域的业务配置人员——所有引擎的配置设计均由该角色执行, STR-E 架构树上无需角色层做分组维度。流程与IT域的其他角色 (业务架构管理员、数据治理员, 见 §六) 属于平台资产治理角色, 不参与 eng 设计路径。

STR-E 的双重身份 (详见 §5.1): 在 A2 视角下, STR-E 是业务配置人员的场景业务 (SL5/PL5 末级节点)——从打开配置页到运行期能力可用的端到端配置业务; 在 A1 视角下, 其对应引擎是 PL3 构件节点, CU 是 STR-E 的细分节点。

与 biz 路径的对称性 (详见 §七): eng 路径的 CU ↔ biz 路径的输出文档 (同为 PL5 末级节点); wft03-eng 的操作页面 ↔ wft03-biz 的配置需求方案 (同属 SR-F 层)。当前约 12 个引擎 → 约 12 个 STR-E, 与 STR-F 下辖输出文档的层级关系完全对称。

8.1 原始需求处理 (wft01-eng)

将零散的 FR-ENG OR 组织为结构化的引擎场景业务方案, 供业务配置人员确认理解。



任务清单:

#	任务	说明
1	推断生成引擎 场景业务节点	接收 FR-ENG OR → 判定归属引擎 → 匹配已有 STR-E 或新建
2	名称级推断场 景业务所需 CU 候选	对称于 STR-F 下辖的输出文档——场景业务由哪些 CU 组成
3	基于能力可交 付性补充引擎 需求	逐 STR-E 检查：业务配置人员完成配置后，该引擎能力是否形成可交付、可应用的端到端闭环——业务人员从打开业务页面到使用该能力获得运行期产出，全链路是否完整。检出缺口 → 生成补充 FR-ENG 材料；仅"配置完成后怎么算能力可用"不明确 → 标记澄清项，不生成补充材料
4	建立追溯链	FR-ENG OR → 引擎 → CU 候选

8.2 业务架构开发 (wft02-eng)

将名称级 CU 候选展开为详细的配置单元设计方案——A2 视角描述业务配置人员怎么工作，A1 视角描述引擎怎么被配置。

输入 STR-E 节点 (可以分解分配 / 待补充分解分配)	产出 A1 配置单元 BA 详细设计 (可以SR设计) 末级节点: 配置单元 (CU), 定义见 §3.3 和 §4.3 细分: 操作页面 (配置页面 + 业务页面)
--------------------------------------	--

任务清单：

#	任务	视角	说明
1	装配 A2 产品业务 架构	A2	业务配置人员 → STR-E 场景业务 → 操作页面序列。描述业务配置人员从打开引擎配置入口到运行期能力可用的端到端配置 workflow
2	装配 A1 产品业务 架构	A1	接收 STR-E → 按引擎→CU 层级展开
3	展开 CU 详细设计	A1	配置要素 / 系统处理说明 / 运行期产出 / 引擎状态迁移
4	为每个 CU 设计操 作页面	—	配置页面 + 业务页面的设计方案
5	基于 A1 补充引擎 需求	—	检出 CU 缺口 → 生成 FR-ENG 线索或补充 OR 材料

A2/A1 双视角：Task 1 对应 §5.2 所述流程与IT域的 A2 产品——描述业务配置人员"怎么工作"（从菜单入口→配置页序列→能力可用）；Task 2~3 对应 §5.1 所述 A1 产品——描述引擎"怎么被配置"（引擎→CU 层级→配置要素）。两者通过操作页面交汇：同一张配置页，在 A2 侧是业务配置人员的操作界面，在 A1 侧是 CU 的配置入口。

CU 的两类页面：配置页面为硬代码实现——配置人员编辑 CU 的配置要素，提交后引擎校验并保存；业务页面由引擎按 CU 定义渲染——在操作页面组件上渲染构件能力（见 §4.3）。

8.3 系统需求开发 (wft03-eng)

为每个操作页面识别所需构件（展示/交互/数据/反馈类），产出引擎功能 SR-F 需求。对应 biz 路径的 wft03-biz，同属 SR-F 层。

输入
BA 节点 (A1)
(可以SR设计 / 待补充SR设计) —→

产出
引擎功能 SR-F 需求 (待产品架构)
末级节点: 操作页面
细分: 构件 (详见 §4.2)

任务清单:

#	任务	说明
1	为每个操作页面推断所需构件清单	逐操作页面识别所需的展示类/交互类/数据类/反馈类构件
2	生成功能架构 (SR-F)	基于操作页面 + 构件组合形成功能架构
3	为构件推断交互行为	触发方式 / 操作对象 / 结果反馈

8.4 产品架构开发 (wft05-eng)

将构件展开为 PA 组件需求定义——前端组件 + 后端组件 + 平台服务组件, 构件开发流水线可据此进入开发。(eng 路径跳过 wft04——wft04 为 nfr 路径的系统需求开发 Skill, 见 §九。)

输入
SR-F 构件清单 + NFR 约束包
(待产品架构 / 待补充PA设计) —→

产出
PA 组件需求定义 (待追溯验证)
末级节点: PA 组件
细分: 前端组件 / 后端组件 / 平台服务组件

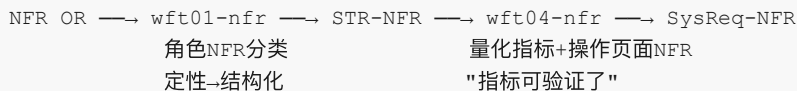
任务清单:

#	任务	说明
1	逐构件判定前端组件需求	构件需要哪些前端组件来实现其 UI 呈现和交互
2	逐构件判定后端组件需求	构件需要哪些后端组件来实现其数据处理和业务逻辑
3	消费 NFR 约束包并制定平台架构决策	3a. 加载约束包 ——从 wft04-nfr 产出中加载 wft05-eng NFR 约束包，按消费版本分流（首次消费→全量加载，版本递增→按变更声明仅处理增量/修订行）。 3b. 约束→组件映射 ——逐约束行判定：CU 级约束 → 对应 CU 的前后端组件设计参数（记入 task 1/2 的组件需求）；PA 组件级约束 → 对应引擎组件的架构参数（并发/缓存/持久化）；全局/业务域/共享约束 → 平台级决策（记入 task 6 的平台服务组件）。 3c. 平台架构决策 ——基于映射结果输出四类平台架构：部署架构（节点/集群/容灾）、通信架构（协议/同步异步/熔断降级）、数据架构（存储/副本/备份）、安全架构（认证/授权/审计）。 3d. 闭合验证 ——逐约束行检查是否已有对应架构决策覆盖，未覆盖的标记「待确认」并给出阻塞原因
4	推断前端组件架构	组件树 / 路由设计 / 状态管理方案
5	推断后端组件架构	API 设计 / 服务划分 / 数据访问方案
6	生成平台服务组件需求定义	将 task 3c 的四类平台架构决策转化为可交付开发的平台服务组件： 部署拓扑组件 （节点规格定义 / 集群编排 / 自动扩缩容策略 / 容灾切换方案）、 通信中间件组件 （API 网关路由与限流 / 消息队列与主题 / 服务注册发现）、 数据基础设施组件 （主库与只读副本 / 缓存集群 / 分片策略 / 备份恢复）、 安全基础设施组件 （认证授权服务 / 密钥与证书管理 / 审计日志采集与告警）。逐组件标注：所承接的 NFR 约束行 ID、实现策略（自研/外购/云服务）、依赖的其他平台服务组件

组件类型：前端组件——UI 层的代码实现单元（组件树/路由/状态管理）；后端组件——服务层的代码实现单元（API 设计/服务划分/数据访问）；平台服务组件——基础设施层的可部署单元（部署拓扑/通信中间件/数据基础设施/安全基础设施），承接 NFR 约束包中的全局/共享/业务域级约束，被所有引擎的前后端组件消费。

九、非功能需求之系统设计

nfr 路径将非功能原始需求 (NFR OR) 逐级转化为可验证的量化指标和可消费的约束包。与 biz/eng 路径并行但设计对象不同——biz 设计业务场景, eng 设计平台引擎能力, nfr 设计横切两者的非功能质量约束。两个 Skill 形成设计接力链:



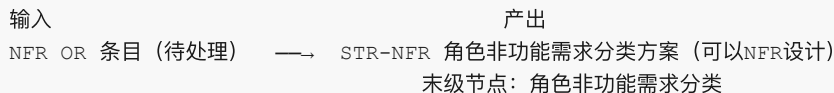
Skill	核心命题	输入→产出	末级节点	交付确认→下游
wft01–nfr	向 NFR 相关方证明"我听懂了你的质量诉求"	NFR OR 条目 → STR–NFR 角色非功能需求分类方案	角色非功能需求分类	NFR 相关方确认 → wft04–nfr
wft04–nfr	产出可验证的指标和可消费的约束包	STR–NFR 节点 → SysReq–NFR 架构 + 量化约束 + 操作页面 NFR	非功能需求分类	wft05–eng / 追溯验证

22 号资产：即 NFR 分类资产，定义非功能需求的一级类目与分类维度、具体需求模板，是 wft01-nfr 判定分类路径和 wft04-nfr 量化设计、架构组织、资产落账的权威参照。nfr 路径两个 Skill 对 22 的写回见 §10.6 参数表。

相关方需求架构文档 02-*.md 在 wft01-nfr 产出 STR-NFR 后形成。SysReq-NFR 写入 05-*.md，06 详细定义在 wft04-nfr 落账时写入 06-*.md。NFR 约束全部流向 eng 路径（能力链）——性能/安全/可用性等非功能质量约束由平台的引擎层保障，不进入 biz 路径（业务配置链）。

9.1 原始非功能需求处理 (wft01-nfr)

定位：nfr 路径的第一个 Skill，从零散但规范化的 NFR OR 到结构化的角色非功能需求分类方案。核心使命是向 NFR 相关方证明“我听懂了你的质量诉求”——识别角色语境、判定 22 分类路径、形成分类需求清单，供 wft04-nfr 展开量化设计。



STR-NFR 的归组锚点为四元组——**用户角色**（提出或承受该非功能诉求的角色/外部系统/保障者）+ **业务/平台语境**（业务域/产品类型/STR-F或STR-E线索）+ **NFR 分类维度**（22 资产中的一级类目+分类维度）+ **适用对象线索**（可能约束的 STR-F/STR-E/业务域/平台能力/全局对象）。

任务清单：

#	任务	说明
1	推断角色 NFR 分类节点	识别 OR 背后的用户角色、业务域/平台能力语境、NFR 一级类目与分类维度，形成 STR-NFR 末级节点
2	推断 NFR 分类需求清单	将 OR 中的质量诉求归纳为分类需求项——定性期望、适用范围线索、来源类型（OR明示 / OR隐含 / 同类语境推断 / 资产线索推断）、待量化线索。wft01 只记录 OR 中的数值/比较词/验证条件， 不将其转为系统化指标 （量化由 wft04 负责）
3	对齐 22 分类资产	逐分类需求项判定 22 分类维度的 Q1/Q2/Q3/Q3+ 关系：Q1→引用已有维度；Q2→引用 + 扩展说明；Q3→新增分类维度建议（标记 [需裁决]）；Q3+→新增一级类目 + 分类维度建议。22 判定全量标注 [推断]，定位为供 wft04 校验的候选建议
4	基于需求清单补充 NFR 需求	闭合检查——检出分类缺口、适用范围缺口、验证语义缺口→生成补充 NFR OR 材料，登记到 OR 原料状态表
5	保持路径边界	将业务功能诉求和平台能力诉求路由到 biz/eng 路径（退回 wft01-biz / wft01-eng），避免 NFR 路径替代功能设计

9.2 系统非功能需求设计（wft04-nfr）

定位：nfr 路径的第二个 Skill，将 STR-NFR 的定性分类需求转化为系统侧 SysReq-NFR 架构——量化指标 + 分层约束表 + 操作页面 NFR。核心使命是**产出可验证的指标和可消费的约束包**，供 eng 路径和追溯验证消费。

输入 STR-NFR 节点 （可以NFR设计 / 待补充NFR设计）	产出 SysReq-NFR 架构 + 量化约束 + 操作页面 NFR （STR-NFR → 已NFR设计，SysReq-NFR → 已验证冻结） 末级节点：非功能需求分类 细分：具体非功能需求
---	---

量化指标的六要素：指标名称 / 目标值与单位 / 统计口径（时间窗口/样本范围/百分位） / 测量方法（压测/审计检查/日志统计等） / 验收条件 / 来源 STR-NFR 与消费版本。模糊诉求处理原则：可依据 22 具体需求模板和同类场景推断时标记 [需裁决] 并给出依据；无法推断时保留 待确认，不推进写回。

任务清单：

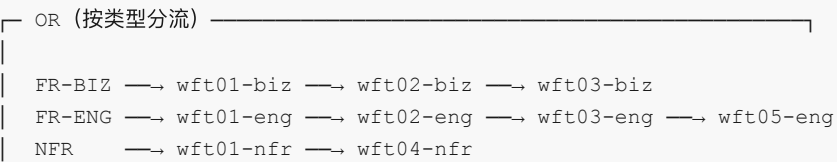
#	任务	说明
1	消费 STR-NFR 分类需求清单	逐 STR-NFR 检查消费版本——首次消费→全量设计，版本递增→按变更声明执行增量/修订/结构型分流。分类边界或角色语境不足→退回 需wft01修订
2	生成量化指标	逐需求项提取待量化线索→匹配 22 具体需求模板和同类场景→生成目标值/单位/统计口径/测量方法/验收条件。信息不足时标注 待确认，不伪造确定值
3	生成非功能需求架构	按 22 分类维度 + 约束对象 + 指标口径匹配或新建 SysReq-NFR 节点，保证每条需求项有唯一系统侧宿主
4	生成分层约束表	逐量化指标判定约束范围（全局 / 业务域 / SR-F / CU / PA组件 / 操作页面）→适用链路（biz / eng / 共享）→约束对象→消费目标→影响级别（阻断型 / 非阻断型 / 待判定）→影响点
5	生成操作页面 NFR	影响用户可感知页面行为的指标→绑定六类页面类型之一 + 约束内容 + 量化目标 + 测量方法 + 消费目标。覆盖页面性能/安全/易用性/可访问性/可恢复性/可观测性六个维度
6	装配 wft05-eng NFR 约束包	按约束范围组织为 wft05-eng NFR 约束包：全局约束 / 业务域约束 / SR-F 约束 / 操作页面约束 / CU 约束 / PA 组件约束 / 共享约束。约束包只引用分层约束表行 ID 和操作页面 NFR 条目，不复制正文。NFR 约束仅流向 eng 路径——非功能质量由引擎层保障，biz 路径不消费
7	闭环检查与资产落账	七项闭环检查（量化完整/宿主唯一/分层正确/页面可消费/biz-eng一致/22可落账/06可验证）→22 具体需求写回→06 系统需求详细定义→05/02 状态推进→STR-NFR 消费版本记录

十、EOS系统设计各Skill

本章定义 EOS 平台系统设计的 Skill 体系——三路径（biz/eng/nfr）自闭环模型及各 Skill 的职责定位和链路关系。biz 路径详见 §七，eng 路径详见 §八，nfr 路径详见 §九。每个 Skill 的内部操作流程和约定见附录A。

10.1 三路径全景

OR 按类型分流至三条路径，每条路径内的 Skill 形成串行接力链——上游 Skill 产出节点，下游 Skill 消费节点后展开为下一层节点。全链路最终产出两套可交付构件开发的成果。各路径 Skill 清单见 §10.3~§10.5，完整参数见 §10.6。



全链路执行顺序

准备阶段

OR 归一化 (ort/pre Skill) → OR 条目状态 = 待处理
NFR OR 同理

第一轮：三路径并行启动（无相互依赖）

biz 线：FR-BIZ OR → wft01-biz → wft02-biz → wft03-biz → 业务配置需求方案
eng 线：FR-ENG OR → wft01-eng → wft02-eng → wft03-eng → (等待 NFR 约束包)
nfr 线：NFR OR → wft01-nfr → wft04-nfr → NFR 约束包

第二轮：eng 线收尾（依赖 nfr 线产出）

eng 线：NFR 约束包就绪 → wft05-eng → 平台 PA 组件需求定义

两条最终产出线

产出线	路径	终点 Skill	最终产出	消费方
业务配置架构及构件需求定义	biz	wft03-biz	功能表单需求定义 + 操作页面需求定义 + 引擎能力缺口线索	构件开发流水线
平台产品架构及构件需求定义	eng	wft05-eng	PA 组件需求定义（前端+后端+平台服务组件，含 NFR 约束消费）	构件开发流水线

各 Skill 的统一运行方式见 §10.2，内部操作流程和约定见附录A。

10.2 Skill 运行方式

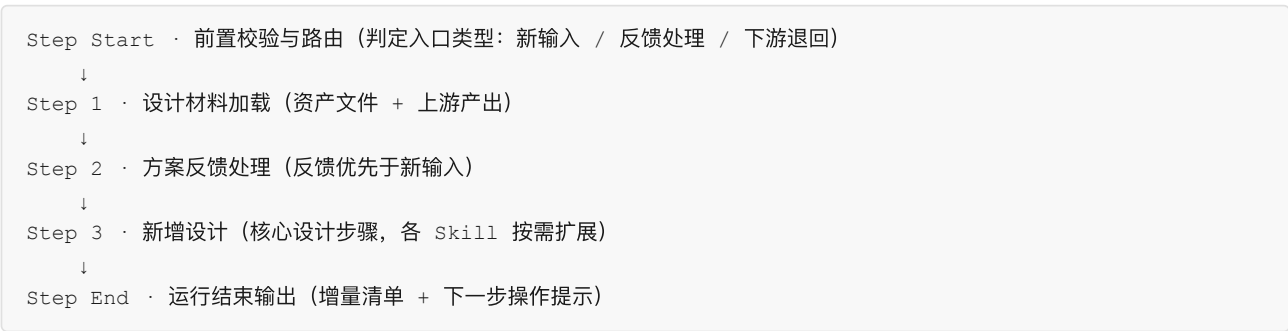
各 Skill 的运行方式统一——相同的启动方式、相同的反馈交互协议、相同的状态推进规则。本节说明人类如何操作单个 Skill、AI 内部如何处理、以及多次重入的场景和目的。详细协议见附录A。

统一运行模型

每个 Skill 对人类而言只有三个动作：

动作	时机	操作
① 启动	上游产出就绪，输入节点状态符合消费条件（见 §10.6 各 Skill 输入合法状态）	在 IDE 中选中输入节点，调用 Skill
② 审阅	AI 产出方案后（节点含 [新增] / [变更] 标记，确认状态为空）	逐条标注 [同意] / [修改] / [驳回]，或回复「整体确认」
③ 重入	标注后再次调用 Skill，AI 按 §A.5 反馈交互协议处理标注	Skill 自动检测反馈 → 处理 → 追加 [已处理] → 推进状态

Skill 被调用后，AI 内部按以下流程自动执行，人类无需干预：



实际运行中，人类通常经历两轮：第一轮启动 Skill 获得方案 → 审阅标注 → 第二轮重入 Skill 处理反馈并推进状态。一轮确认完毕即可；若有反复修改，多轮重入即可，协议相同。

Skill 的连续重入

各 Skill 不是一次性工具——同一 Skill 可在流水线运行期间被多次调用。每次重入时 AI 按 §A.4 入口协议自动判定触发原因并执行对应处理。人类无需区分重入原因，只需在以下场景发生时再次调用 Skill：

重入场景	触发条件	目的
反馈迭代	人类在方案条目上标注了 [修改] / [驳回]，尚未全部 [已处理]	修订方案至人类满意，推进至可推进态
新输入增量	上游产出新一批输入节点（如新 OR 条目、新 STR-F 节点），状态符合消费条件	增量展开新节点，已有方案不受影响
下游退回	下游 Skill 发现方案缺陷，将节点退回（状态 = 需 wftXX 修订），附带缺失说明	按人类改进方案修订后重新推进至可推进态
上游变更级联	上游节点因新输入修订（版本递增），下游 Skill 重入时检测到消费版本落后	按变更类型（增量/修订/结构型，见 §A.7）处理受影响的下游节点，保持全链路一致

同一 Skill 可能在同一批 OR 处理期间被调用 2~5 轮（反馈迭代），也可能在几小时到几周的时间跨度内被调用数十次（新输入分批到达、上游变更级联），每次处理一小批节点，逐步累积形成完整的架构方案。

跨路径依赖与并行策略

依赖关系	说明	策略
eng 线 wft05-eng ← nfr 线 wft04-nfr	wft05-eng 消费 NFR 约束包	eng 线前三个 Skill 先行完成, wft05-eng 等 nfr 线产出约束包后启动
eng 线 ← biz 线 wft03-biz	wft03-biz 产出引擎能力缺口线索, 转入 eng 路径	非阻塞——由人类决定何时以线索启动对应 eng Skill

实际建议：三路径可同时启动第一轮。biz 线最短（3 Skill），eng 线前段（3 Skill）与 nfr 线（2 Skill）可并行。全部第一轮完成后，启动 wft05-eng 收尾。

10.3 biz 路径 Skill

Skill	末级节点	细分节点	职责定位	所属章
wft01-biz	场景业务	PL5 输出文档	基于 OR 推断定义场景业务节点和输出文档（含任务模式与主要节点，业务语言），基于端到端补充业务需求——"向业务方证明我听懂了"	§七
wft02-biz	输出文档	PL6-A 活动节点 / PL6-B 正文条目	生成 A2+Bn 产品业务架构；将 wft01 的业务节点转化为 PL6-A 节点需求定义（架构语言）——"设计方案"	§七
wft03-biz	功能表单需求 / 操作页面需求	—	消费 wft02 的 BA 方案→生成功能表单需求定义（字段/布局/交互/展示）+ 操作页面需求定义（页面序列/流转规则/交互需求）+ 引擎能力缺口线索——"出配置需求"	§七

10.4 eng 路径 Skill

Skill	末级节点	细分节点	职责定位	所属章
wft01-eng	场景业务 (STR-E)	CU 候选（名称级）	基于 FR-ENG OR 推断引擎场景业务 + 名称级 CU 候选——"向平台方证明我听懂了"	§八
wft02-eng	配置单元 (CU)	操作页面（配置页+业务页）	装配 A2（业务配置人员→STR-E→操作页面序列）+ A1（引擎→CU 层级）+ CU 详细设计 + 操作页面方案——"设计方案"	§八
wft03-eng	操作页面	构件	逐操作页面识别所需构件（展示/交互/数据/反馈类），生成 SR-F 构件清单——"配齐构件"	§八
wft05-eng	PA 组件	前端组件 / 后端组件 / 平台服务组件	逐构件判定前后端组件需求 + 推断平台/前端/后端三层架构——"出组件需求"	§八

10.5 nfr 路径 Skill

Skill	末级节点	细分节点	职责定位	所属章
wft01-nfr	角色非功能需求分类	—	基于 NFR OR 推断角色 NFR 分类节点和需求清单，对齐 22 分类资产，基于需求清单补充 NFR 需求——"向 NFR 相关方证明我听懂了你的质量诉求"	§九
wft04-nfr	非功能需求分类	具体非功能需求	消费 STR-NFR → 生成量化指标 + NFR 架构 + 分层约束表 + 操作页面 NFR + 下游约束包 → 闭环检查与资产落账——"产出可验证的指标和可消费的约束包"	§九

10.6 全量 Skill 参数速查

以下为各 Skill 关键参数的汇总速查。各 Skill 的完整参数定义在其 Skill 文档的第一章和第三章 Step Start 中。

biz 路径：

维度	wft01-biz (OR→STR-F)	wft02-biz (STR-F→BA)	wft03-biz (BA→配置需求方案)
末级节点	场景业务 (STR-F)	输出文档 (PL5)	功能表单需求定义 / 操作页面需求定义
输入对象	FR-BIZ OR 条目 (≤10)	STR-F 节点 (≤5)	BA 节点 (≤5)
输入合法状态	待处理	可以分解分配 / 待补充分解分配	可以SR设计 / 待补充SR设计
产出状态	可以分解分配	可以SR设计	可构件开发
下游 Skill	wft02-biz	wft03-biz	构件开发流水线
退回状态	需wft01修订	需wft02修订	需wft03修订
资产写回	23 (Bn 对齐引用)	23/24/26/27/28	04/05/06
补充材料	补充 OR 材料 (闭合缺口)	FR-BIZ OR 材料 (文档缺口)	FR-ENG 线索 (引擎缺口)

eng 路径：

维度	wft01-eng (OR→STR-E)	wft02-eng (STR-E→BA)	wft03-eng (BA→SR-F)	wft05-eng (SR-F→PA)
末级节点	场景业务 (STR-E)	配置单元 (CU)	操作页面	PA 组件 (前端/后端/平台服务)
输入对象	FR-ENG OR 条目 (≤10)	STR-E 节点 (≤5)	BA 节点 (A1, ≤5)	SR-F 构件清单 + wft05-eng NFR 约束包
输入合法状态	待处理	可以分解分配 / 待补充分解分配	可以SR设计 / 待补充SR设计	待产品架构 / 待补充PA设计
产出状态	可以分解分配	可以SR设计	待产品架构	待追溯验证
下游 Skill	wft02-eng	wft03-eng	wft05-eng	构件开发流水线
退回状态	需wft01修订	需wft02修订	需wft03修订	需wft05修订
资产写回	01/02/23/25	02/04/23/25	04/05/25	05/06/07/23/25
补充材料	补充 FR-ENG 材料 (能力链缺口)	FR-ENG 线索或补充 OR 材料	—	—

nfr 路径：

维度	wft01-nfr (OR→STR-NFR)	wft04-nfr (STR-NFR→SysReq-NFR)
末级节点	角色非功能需求分类	非功能需求分类 / 具体非功能需求
输入对象	NFR OR 条目 (≤10)	STR-NFR 节点 (≤5)
输入合法状态	待处理	可以NFR设计 / 待补充NFR设计
产出状态	可以NFR设计	STR-NFR → 已NFR设计, SysReq-NFR → 已验证冻结
下游 Skill	wft04-nfr	wft05-eng / 追溯验证
退回状态	需wft01修订	需wft04修订
资产写回	02/22	05/06/22/02
补充材料	补充 NFR OR 材料	上游退回说明 / 指标待确认项

十一、产品文档结构、读写及反馈

适用范围： 0107—2428 号产品数据文档与资产文档。08（追溯矩阵）和 09（验证报告）保留原格式，不适用本章规范。适用于编写和维护这些文档的人类和 AI。

本章定位： 91 规范 §七~§十 定义"怎么设计"，本章定义"产出物怎么组织、怎么读写、怎么反馈"——每个产品数据文档的物理结构、索引机制、节点格式、AI 读写协议以及人类反馈方式。

11.1 文档全景

每个产品数据文档由以下区域从上到下按固定顺序组成：

```
└─ 元数据 ───────────
| # EOS {文档名称}
| **文档版本**：v{版本}
| **修订日期**：{日期}
| **HEAD @上次 AI 运行**：{commit-hash}
|
└─ 自描述索引 ───────────
| 索引行数：{N}
|
└─ 业务域索引 ───────────
| | 域 | 节点数 | 入口 |
|
└─ AI最近变更 ───────────
| AI 写回记录表，含超链接
|
└─ AI可以处理节点 ───────────
| 按 Skill 分节列出待处理节点
|
└─ 域正文块1 ───────────
| ## {域名} {#anchor}
| ### 域产品索引
| ### 产品正文块 / {E2E 项目名}
|     ##### {构件名称}
|     ##### @{id} 节点
|     .....
| ### 产品正文块 / {下一个 E2E 项目名}
|     ##### {构件名称}
|     .....
|
└─ 域正文块2（下一个域）
└─ ...
```

适用本章规范的文件清单：

编号	文件名	说明
01	01-eos-original-requirements.md	原始需求
02	02-eos-stakeholder-requirements-architecture.md	相关方需求架构
03	03-eos-stakeholder-requirements-detailed.md	相关方需求详细
04	04-eos-business-architecture.md	业务架构
05	05-eos-system-requirements-architecture.md	系统需求架构
06	06-eos-system-requirements-detailed.md	系统需求详细
07	07-eos-product-architecture.md	产品架构
21	21-eos-stakeholder-roles.md	角色定义
22	22-eos-nfr-taxonomy.md	NFR 分类资产
23	23-eos-output-architecture.md	输出产品架构
24	24-eos-business-assets.md	可复用业务资产
25	25-eos-engine-models.md	引擎资产
26	26-eos-document-definitions.md	文档资产
27	27-eos-resource-assets.md	资源资产
28	28-eos-dashboard-indicator-assets.md	看板指标资产

08（追溯矩阵）和 09（验证报告）不适用本章结构，保留原有格式。34 和 35 为中间过程文档，不适用本章规范。

11.2 自描述索引与业务域索引

文件头部的索引系统是 AI 导航文档的唯一入口。

为什么不用行号索引？ 产品数据文档由人和 AI 共同编辑。人类在文档中插入或删除行后，原行号全部失效。索引表无法预测“哪一行对应哪个节点”——因此索引表不存行号，AI 通过 `grep -n '@{节点ID}'` 实时定位节点的当前行号。人类插删内容后，grep 仍然能找到节点，不受行号漂移影响。

自描述索引（索引行数：{N}）告诉 AI 业务域索引表有几行：

```
## 自描述索引
索引行数：4

## 业务域索引
| 域 | 节点数 | 入口 |
|----|-----|-----|
| 采购管理 | 420 | [→] (#procurement) |
| 生产管理 | 530 | [→] (#production) |
| ... | ... | ... |
```

部分	说明
自描述索引	索引行数：N，N 为 ## 业务域索引 表格的总行数（含表头）
业务域索引	列出所有业务域/角色组（约 15 行），每行标注节点数和锚点入口。常驻 AI context

文档元数据位于文件第一段，记载版本和 AI 运行追踪信息：

字段	说明	维护方
文档版本	语义版本号	AI 每次运行结束时递增
修订日期	最近一次修改日期	AI 每次运行结束时更新
HEAD @上次 AI 运行	AI 上次提交时的 commit hash	AI 每次运行结束时自动写入

HEAD @上次 AI 运行 由 AI 自动维护。人类无需手动修改；若因人为编辑导致不准确，AI 在 Step Start 中自动修正。

11.3 域级组织

每个业务域/角色组在文档中占据一个独立节，结构如下：

```
## {域名称} {#anchor}

### 域产品索引
| 产品 | 节点数 | 待裁决 |
|-----|-----|-----|
| {E2E 项目名} | {节点数} | {待裁决数} |

### 产品正文块 / {E2E 项目名}
#### {构件名称}
##### @{id} 节点名称
...
```

规则	说明
R1	每个 ## {域名称} 必须有锚点 ID，与业务域索引的入口列对应
R2	域产品索引只索引到 E2E 项目（产品）级，不索引到构件
R3	### 产品正文块 标题带 E2E 项目名（### 产品正文块 / {E2E 项目名}），与域产品索引的产品列对应。正文块内部按构件→WBS→输出文档→节点逐层用 ##### / ##### 展开

11.4 节点块

所有末级节点统一使用 ##### @{id} 块：

```
##### @{id} 节点名称
**类型**: {层级}.{子类}
**父节点**: @{parent-id}
**输入**: @{input-doc-id} / {描述}
**输出**: @{output-doc-id} / {描述}
**子条目状态**: x 待处理 / y 已确认 / z 需裁决 (有子条目时写此项, 无子条目时省略)
**确认状态**: [同意] / [修改] / [需裁决] / (空)
```

公共字段 (所有文件共用):

字段	格式	说明
类型	{层级}.{子类}	如 BA.L5、SR-F.L3
父节点	@{id}	父节点 ID
输入	@{id} 或自由文本	多个用 / 分隔
输出	@{id} 或自由文本	同上
子条目状态	x 待处理 / y 已确认 / z 需裁决	有子条目时必须填, 汇总子条目的确认状态分布; 无子条目时省略
确认状态	[同意] / [修改] / [需裁决] / (空)	子条目的确认状态不记入本字段, 由 **子条目状态** 汇总

各文档特有字段 (如 BA 的 **支撑引擎**、SR-F 的 **验收条件** / **量化指标**) 定义在 91 规范各对应章节中。

11.5 ID 命名与跨文件引用

ID 命名规则:

前缀	所属	示例
@ba-	BA 节点	@ba-proc-apply-001
@srf-	SR-F 节点	@srf-proc-apply-001
@srn-	SR-NFR 节点	@srn-security-001
@doc-	文档/信息载体	@doc-purchase-request
@engine-	引擎	@engine-flow
@node-	构件类型	@node-proc-apply
@prod-	E2E 项目	@prod-procurement
@domain-	业务域	@domain-procurement

跨文件引用规则:

规则	说明
R4	跨文件引用通过 @id，不包含文件路径
R5	@id 全局唯一，前缀确保不冲突
R6	AI 遇到未知 @id 时，按前缀判断归属文件，从该文件业务域索引中查找

11.6 AI最近变更

每个产品数据文档在AI可以处理节点之前包含一个 ## AI最近变更 节，记录 AI 最近对该文档的写回操作。人类打开文档后可快速了解 AI 最近做了什么、影响到哪些节点。

条目格式：

```
## AI最近变更

| 时间 | 操作 Skill | 操作类型 | 节点 ID | 变更摘要 |
|-----|-----|-----|-----|-----|
| 2026-07-11 | wft02-biz | [变更→] (#@ba-proc-apply-001) | @ba-proc-apply-001 | 采购申请：审批节点从2级改为3级 |
| 2026-07-11 | wft02-biz | [新增→] (#@ba-proc-order-005) | @ba-proc-order-005 | 采购订单：新增创建采购订单流程 |
```

列	说明
时间	AI 写回日期
操作 Skill	执行写回的 Skill 名称
操作类型	新增 / 变更 / 删除 / 确认处理，用 [->] (#节点ID) 超链接，点击跳转到对应节点
节点 ID	被操作的节点 ID
变更摘要	一句话说明变更内容

维护规则：

- 由 AI 在每次写回时（§11.11）在文件头的 ## AI最近变更 表格中追加一行。
- 保留最近 30 条变更。超出时从最旧的行开始滚动删除。
- 人类不直接修改此表。AI 在写回时自动维护。

规则	说明
R7	每次写回完成后必须追加一条AI最近变更记录。变更记录的 节点 ID 列必须包含到对应节点的文档内超链接
R8	AI最近变更记录保留最近 30 条。溢出时删除最旧记录

11.7 AI可以处理节点

AI可以处理节点按 Skill 名称分节列出所有当前可被处理的节点，由各 Skill 写回时自动维护。

标准分节：

分节	节点条件	适用 Skill
待 ort03 处理	原始需求状态 = 待处理	ort03
待 wft01-biz 处理	FR, 状态 = 待分解映射 / 待确认方案 / 迭代中	wft01-biz
待 wft01-eng 处理	FR-ENG, 状态 = 待处理	wft01-eng
待 wft01-nfr 处理	NFR, 状态 = 待处理 / 待确认方案 / 迭代中	wft01-nfr
待反馈处理	确认状态 = [修改] / [驳回] 且无 [已处理] ; 有子条目时 **子条目状态** 含"待处理"	所有 Skill
待下游退回处理	状态 = 需wftXX修订	所有 Skill

条目格式：

```
### 待 wft01-biz 处理
| 节点 ID | 标题 | 优先级 | 当前状态 | 所属域/角色 |
|-----|-----|-----|-----|-----|
| @ba-proc-apply-001 | 创建采购申请 | P0 | 待分解映射 | 采购管理 |
```

规则	说明
R7	AI可以处理节点只列出节点级条目。子条目状态通过父节点 **子条目状态** 汇总，不单独列出
R8	同一节点同时满足多个分节条件时，按优先级取最高一节（ort03 > biz > eng > nfr > 反馈 > 退回）

AI可以处理节点的分节由 §10.6 定义的输入合法状态驱动。

11.8 人类反馈方式

人类对 AI 产出的方案进行反馈，有以下几种方式和对应的字段：

一、结构化反馈（推荐）

在节点块的 **确认状态** 字段中直接标注：

标注	含义	AI 处理
[同意]	接受该节点/条目的设计	锁定该节点，推进至可推进态
[修改]：{修改内容}	接受方案但需调整	按修改内容修订后标记 [已处理]
[驳回]：{驳回理由}	方案不可接受，需重新设计	检查需求缺口，退回重做后标记 [已处理]
[需裁决]：{待决事项}	涉及架构边界裁定，AI 无法自主决定	等待人类仲裁意见

对有子条目的节点，人类可以在父节点上整体标注，也可以在各个子条目上单独标注：

```
-- 父节点级标注（影响整个节点） --
##### @ba-proc-apply-001 创建采购申请
...
**确认状态**：[同意]    ← 整个采购申请节点被确认

-- 子条目级标注（仅影响该子条目） --
##### @ba-proc-apply-001-sub-01 物料需求汇总
...
**确认状态**：[修改]：物料编码格式改为10位    ← 仅该条目需修改
```

子条目级标注的汇总由 ****子条目状态**** 字段自动反映到父节点。

二、非结构化反馈（AI 兜底处理）

人类可以在节点块的任何位置写自然语言意见：

```
##### OR-008 权限分级管控
**优先级**：P0
**原始表述**：三级权限分级管控
这里的分级不对，应该是四级    ← 自由文本，非结构化
```

AI 通过 `git diff` (§11.10) 检测到此类变更后，尝试理解意图并标记 **[需确认]**，提交人类审核。

节点块以外的反馈（如文档头签名、尾注、文件中任意位置的批注）同样通过 `git diff` 检测，AI 自动识别其所在节点归属后处理。

三、批量确认

当人类对一批节点方案整体满意时，可以一次性确认：

方式	操作	效果
整体确认	回复「整体确认」	AI 对当前方案中所有 **确认状态** 为空的节点写入 [同意]
批量标注	在连续的多个节点块上标注相同标记	AI 按连续行的范围统一处理

四、反馈后的处理流程

```
人类标注 → AI 加载确认状态 → 处理反馈 → 同步更新：
├── 节点 **确认状态**
├── 父节点 **子条目状态**（子条目标注时）
├── AI可以处理节点（移出已处理节点）
└── git commit（记录本轮反馈处理结果）
```

规则	说明
R9	人类优先在 **确认状态** 字段中用结构化标注反馈。非结构化反馈经 AI 判定后标记 [需确认]
R10	子条目标注优先级高于父节点标注。父节点标注多条子条目时，子条目标注各自独立处理
R11	所有标注的处理结果（ [已处理] ）由 AI 在反馈处理后写入，人类无需手动标记

11.9 AI 读取规则

AI 读取文档时有两种入口，互不替代：

入口 A: AI可以处理节点（用于 Step Start 入口判定）

AI 判定"有什么可以处理"时，读AI可以处理节点：

```
1. Bash("grep -n '待{Skill名称}' 目标文件") → 定位就绪分节
2. Read 该分节 → 获取待处理节点 ID 列表
3. 有节点 → 逐节点: Bash("grep -n '@{节点ID}' 目标文件") 定位行号
   → Read 节点块
```

入口 B: 索引导航（用于按域/角色浏览）

AI 要查看某域/角色下有哪些节点时，按索引导航：

```
1. Bash("grep -n '索引行数: ' 目标文件") → 定位索引行号 L
2. 从 L 提取 N → Read 业务域索引/角色索引
3. 按锚点 Read 域内索引 → 获节点 ID 列表
4. 按需: Bash("grep -n '@{节点ID}' 目标文件") → 定位行号 → Read 节点
```

两者分工：

	AI可以处理节点	索引导航
用途	Step Start 发现"有什么可以处理"	浏览"某域/角色有什么节点"
驱动	Skill 重入时自动执行	人类询问或跨域引用时执行
场景	新输入/反馈/退回入口判定	按域浏览、定位引用、检查跨域

规则	说明
R12	AI 不得遍历索引表以外的区域搜索节点。索引表是发现节点存在的唯一入口
R13	索引表不存储行范围。节点行号通过 <code>grep -n '@{节点ID}'</code> 实时定位

11.10 AI 反馈检测规则

为什么用 git diff? 人类的反馈方式不限于结构化标注（[同意] / [修改] / [驳回]），也经常在文档中直接写自由文本（如"这里逻辑不对，应该先审批再创建"）。AI 无法提前预测"哪里被碰过"——但 git diff 可以精确回答"上次 AI 运行后，文档中哪些行变了"。AI 只检查变更行附近的节点，不遍历全文，也不依赖人类按规范格式标注。

AI 在 Step Start 中通过 git diff 检测自上次运行以来人类修改过文档：

```
1. 读文件头 **HEAD @上次 AI 运行** 字段，获上次提交 hash H
2. Bash("git diff H..HEAD -- 目标文件") → H 到当前 HEAD 的全部变更
3. 无变更 → 正常流程
   有变更 → 解析变更行号，读变更行附近节点块（上下 5 行）
   → 判定变更类型：
      - [同意]/[修改]/[驳回] → 按 §A.5 处理
      - 自由文本 → 尝试理解意图，标记 `[需确认]`
      - 格式/排版 → 忽略
   → 将存在反馈的节点加入AI可以处理节点"待反馈处理"分节
```

规则	说明
R14	<code>**HEAD @上次 AI 运行**</code> 由 AI 自动更新为当前提交 hash。人类无需手动修改
R15	若 HEAD 字段不是最近的 AI 提交，AI 在 Step Start 中自动修正
R16	非标反馈经 AI 判定后标记 <code>[需确认]</code>

11.11 AI 写回规则

写入时的节点定位——通过锚点 ID 定位，不依赖行号：

1. <code>Bash("grep -n '@{节点ID}' 目标文件")</code>	→ 定位节点当前行号
2. Read 节点块	→ 确认目标（验证 ID 和名称）
3. Edit 按行号修改节点块内容	
4. 若为新节点： <code>Bash("grep -n '## {域名称}' 目标文件")</code>	→ 定位域分组 在域分组末尾插入新节点

同步更新清单——每次写回必须同步维护以下内容：

操作	同时维护
新增节点	索引表加入节点 ID；AI可以处理节点对应分节加入节点
修改节点状态	AI可以处理节点对应分节加入或移出节点
删除节点	索引表和AI可以处理节点移除节点 ID
新增/修改/删除子条目	更新父节点 <code>**子条目状态**</code> 汇总值
人类标注确认状态	子条目更新同步父节点 <code>**子条目状态**</code> ；父节点标注更新 <code>**确认状态**</code> ；AI可以处理节点"待反馈处理"同步

提交规范——每次写回后必须：

1. 更新文件头 <code>**文档版本**</code> （递增次版本号）
2. 更新文件头 <code>**修订日期**</code>
3. 更新文件头 <code>**HEAD @上次 AI 运行**</code> = 当前提交 hash
4. <code>git add + git commit -m 带上 Co-Authored-By: Claude</code>

规则	说明
R17	写回必须在本 Skill 同一运行步骤内完成全部更新（索引表 + AI可以处理节点 + 子条目状态 + 文件头字段）
R18	节点 ID 是定位节点的唯一标识。不使用行号，不依赖文件的行序号
R19	每次写回后必须提交。未提交的写回无法被下次 git diff 检测到

11.12 按需读取规则

仅以下情况允许扩大读取范围：

情况	操作
节点 ID 在索引表中无法定位	全文件搜索目标 ID，确认归属
需要跨域检查引用一致性	加载目标域节点块
人类明确要求通读	按要求完整加载指定区域

扩大读取完成后，非常驻内容必须从 context 中移除。

禁止行为：

- 不得在常规 Step 运行中默认加载完整文件正文
- 不得将 疑似 或 待确认 节点的 ID 作为下游强依赖

附录A：Skill 布局、规范及约定（规范性附录）

本附录是什么：定义所有 Skill 的统一框架。每个 Skill 有两面——**静态面**（作为一份文档，有固定的五章结构和格式约定）和**动态面**（作为运行单元，遵循入口→反馈→生命周期→传播的行为协议）。各 Skill 文档引用本附录对应节次，并补充本 Skill 的特化规则。

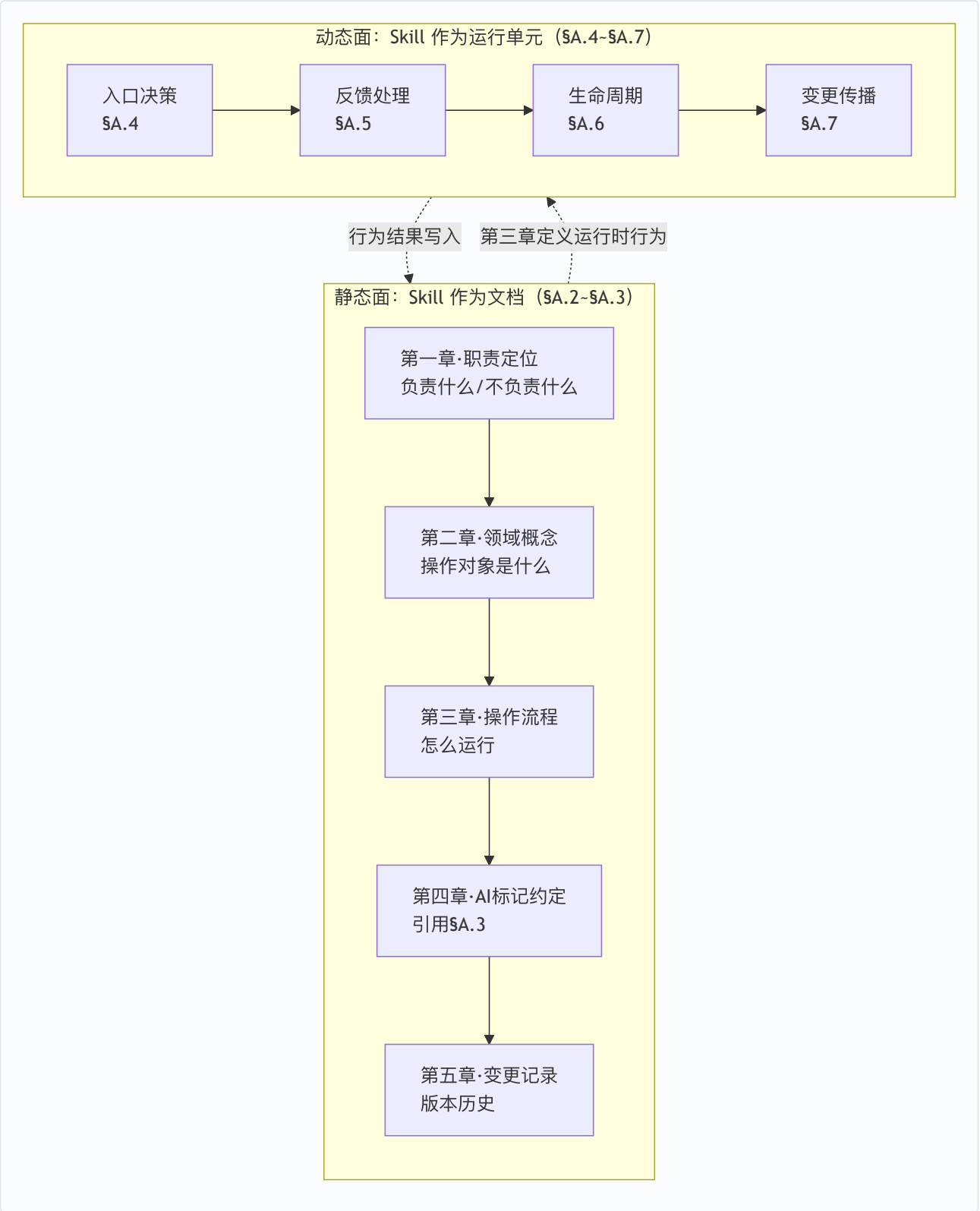
第一部分：导览

A.1 阅读地图

按你的目标选择阅读路径：

你的目标	阅读路径	预估时间
写一个新的 Skill 文档	第二部分（§A.2 五章结构 → §A.3 格式约定）→ 用 §A.8.1 检查清单自检	~15 分钟
理解 Skill 如何运行	第三部分（§A.4 入口 → §A.5 反馈 → §A.6 生命周期 → §A.7 变更传播），按顺序阅读	~20 分钟
运行时遇到具体问题，速查答案	§A.8.2 速查表，按问题跳入对应节次	~2 分钟

全景图——静态面定义"是什么"，动态面定义"怎么运行"：



核心原则：静态面定义"是什么"，动态面定义"怎么运行"，格式约定统一表达方式。写文档时读第二部分，理解运行时读第三部分，需要速查跳 §A.8。

第二部分：编写 Skill 文档

适合谁：正在编写或维护 Skill 文档的人。其他读者可跳至第三部分。

A.2 Skill 文档的五章结构

每个 Skill 文档按五章组织，各章有固定的子节布局和内容约束。

A.2.1 五章总览

章	标题规则	一句话定位	内容约束
一	职责定位	定义"负责什么/不负责什么"	目的与目标 + 上下游衔接
二	{领域概念基础}	定义"操作对象是什么"	只定义概念，不定义操作流程
三	操作流程	定义"怎么做"	Step Start→Step 1→...→Step End
四	AI 标记约定	引用 §A.3 + 本 Skill 示例	格式规范，不含操作逻辑
五	变更记录	记录版本历史	日期/版本/说明 三列表

A.2.2 第一章 · 职责定位

子节	内容	写作要求
1.1 目的与目标	主要目的（引用块）+ 目标（5 条编号列表）+ 本 Skill 负责 + 本 Skill 不负责	目标对齐 91 规范对应节的任务清单；负责/不负责按操作顺序排列，含路由指向
1.2 与上下游的衔接	上游 Skill（产出什么、本 Skill 消费什么）+ 下游 Skill（消费什么、退回机制）	各一段，含 91 规范 § 引用

A.2.3 第二章 · 领域概念基础

要素	规则
标题命名	按本 Skill 的末级节点命名（wft01-biz → 「场景业务设计基础」，wft02-biz → 「BA 设计基础」）
章首引文	列出本章定义的领域对象 + 权威定义源引用 + 操作流程见第三章
子节组织	定义 → 分类 → 架构层级表 → 本 Skill 在该侧的职责
领域规则	设计规则 / 归纳分组 / 覆盖自查清单 放在章末
生命周期	放在规则之后；变更传播协议放在生命周期之后
内容约束	只定义操作对象的「是什么」，不定义「怎么做」

A.2.4 第三章 · 操作流程

子节	内容	写作要求
Step 布局总览	表 (Step 名称 产出 写资产)	章首, 给读者全局地图
Step Start	输入校验→退回优先→反馈处理→新输入路由 (见 §A.4)	校验条件用表格 (校验条件 判定 动作)
Step 1 · 设计材料加载	加载清单 (资产文件 + 用途) + 按需扩大规则	读取/写回协议见 §A.3.3
Step 2 · 方案反馈处理	引用 §A.5 + 本 Skill 特化规则 + 确认状态格式示例	通用协议不重复, 仅写本 Skill 特化部分
Step 3~N	各 Skill 特化设计步骤	按需增减 Step 数量
Step End · 运行结束输出	三区结构 (模板见 §A.3.2)	人类下一步操作提示, 不写入资产

A.2.5 第四章 · AI 标记约定

四类增量标记 ([新增] / [变更] / [确认] / [需裁决]) 的定义及使用规则见 §A.3.1。各 Skill 的第四章引用 §A.3.1, 并补充本 Skill 的增量清单格式示例 (含本层级的完整增量清单), 供人类参照。

A.2.6 第五章 · 变更记录

三列表: 日期 | 版本 | 说明。当前版本写完整说明, 历史版本可折叠 ("详见 Git 历史")。

A.3 格式约定

以下约定为所有 Skill 的统一格式规范。各 Skill 的第四章 (AI 标记约定) 和第三章 Step End 引用本节。

A.3.1 AI 增量标记

每次设计步骤完成后, AI 向人类输出增量清单, 使用四类标记聚焦决策:

标记	含义	示例
[新增]	新创建的节点/条目, 需确认是否需要	[新增] 采购订单审批流程
[变更]	已有条目的修改, 需确认是否同意	[变更] 审批节点从2级改为3级
[确认]	本轮仅补强证据或追加引用, 无实质变化	[确认] 引用23号资产Q1对齐结论
[需裁决]	需人类做架构判断 (边界存疑、矛盾、缺口是否转需求材料等)	[需裁决] 边界存疑—是否纳入质量管理域

补充规则:

规则	说明
资产对齐结果 ≠ 增量标记	Q 级别 (如 23=Q1) 写成 资产对齐: 23=Q1 , 不混入增量清单
[推断] 的使用	不单独作主标记, 只有需人类决策时才标 [需裁决]
输出位置	增量清单在 Step End 输出, 不写入资产文档

A.3.2 Step End 输出模板 (三区结构)

所有 Skill 的 Step End 输出使用统一的三区结构:

当前状态: <状态值>
{本 Skill 特有的结果摘要行}

一、方案反馈

- A. 整体确认 → 回复「整体确认」, 快捷同意全部未标注条目
- B. 局部修改 → 在确认状态中标注 [修改]: ...
- C. 驳回重做 → 在确认状态中标注 [驳回]: ...

二、下一步

本Skill → {本 Skill 的合法重入条件}
后续 → {下游 Skill 及进入条件}

各 Skill 的 Step End 实例化时填充第一区的状态变量和结果摘要行, A/B/C 和"下一步"结构保持不变。

A.3.3 产品数据文档与资产文档读取与写回约定

本约定适用 §十一 所列的全部产品数据文档 (0107) 与资产文档 (2128)。08 (追溯矩阵) 和 09 (验证报告) 保留原格式, 不适用本约定。

一、读取协议

Skill 的 Step 1 (设计材料加载) 加载产品数据或资产文档时, 按以下优先级执行:

步骤零: 变更感知 (检测人类反馈)

读取目标文件之前, AI 先检测自上次运行以来人类是否修改过该文件:

1. 读取文件头 `**HEAD @上次 AI 运行**` 字段, 获取上次提交 hash H
2. `Bash("git diff H..HEAD -- 目标文件")` → 返回 H 到当前 HEAD 之间的全部变更
3. 无变更 → 进入步骤一 (读AI可以处理节点)
 - 有变更 → 解析变更行号, 读取变更行附近的节点块 (上下 5 行)
 - 判定变更内容类型:
 - 结构化标注 ([同意]/[修改]/[驳回]) → 按 §A.5 协议处理
 - 非标自由文本 → AI 尝试理解意图, 标记 `[需确认]`
 - 无关修改 (格式/排版) → 忽略
 - 将存在反馈的节点加入AI可以处理节点的"待反馈处理"分节

规则	说明
R0	AI 每次运行结束后更新文件头的 <code>**HEAD @上次 AI 运行**</code> 为当前提交的 hash
R0a	若 <code>**HEAD @上次 AI 运行**</code> 不是距离 HEAD 最近的 AI 提交, AI 在 Step Start 中自动修正
R0b	非标反馈由 AI 判定后标记 `[需确认]`, 不走AI可以处理节点的结构化标记规则

步骤一: 读AI可以处理节点 (优先)

如果当前 Skill 有对应的AI可以处理节点分节 (§11.7), 先读AI可以处理节点:

1. `Bash("grep -n '待{Skill名称}' 目标文件")` → 定位就绪分节起始行
2. Read 该就绪分节 → 获取待处理节点列表
3. 有节点 → 逐节点: `Bash("grep -n '@{节点ID}' 目标文件")` 定位行号
 - Read 节点块, 无需走索引导航无节点 → 进入步骤二, 按业务域索引加载目标段

步骤二: 索引导航 (AI可以处理节点无待处理时)

1. Bash("grep -n '索引行数: ' 目标文件") → 定位索引行号 L
2. 从 L 提取数字 N (索引表格总行数, 含表头)
3. Read(file, offset=L+1, limit=N) → 加载业务域索引
4. 按目标业务域在业务域索引中查找产品级入口锚点
5. Read 域产品索引 → 获取该域下的节点 ID 列表
6. 按需: Bash("grep -n '@{节点ID}' 目标文件") → 定位行号 → Read 节点

规则	说明
R1	AI可以处理节点优先级高于索引导航。只要当前 Skill 有AI可以处理节点分节, 必须先读AI可以处理节点
R2	角色索引/业务域索引常驻当前 Skill 运行的 context, 直至该文件不再需要
R3	产品正文块处理完毕后, 可从 context 中移除, 角色索引/业务域索引保留

按需扩大规则:

仅在以下情况允许扩大读取范围:

- 目标节点 ID 在角色索引/业务域索引中无法定位
- 需要跨域检查引用一致性
- 人类明确要求通读产品数据文件

禁止行为:

- 不得在常规 Step 运行中默认加载完整文件正文
- 不得将 疑似 或 待确认 节点的 ID 作为下游强依赖

二、写回维护规则

AI 写入或修改产品数据文档或资产文档中的节点后, 必须同步更新以下内容:

操作	更新内容
新增节点	将节点 ID 加入域产品索引与业务域索引/角色索引; 将节点 ID 加入AI可以处理节点对应分节
修改节点	若变更 **当前状态**, 将节点加入或移出AI可以处理节点对应分节
删除节点	从域产品索引与业务域索引/角色索引中移除节点 ID; 从AI可以处理节点中移除该节点
标注 [需裁决]	域产品索引的 待裁决 加 1
取消 [需裁决]	域产品索引的 待裁决 减 1
新增/修改/删除子条目	更新父节点的 **子条目状态** 汇总值 (X 待处理 / Y 已确认 / Z 需裁决); 子条目待处理状态影响父节点是否进入AI可以处理节点的"待反馈处理"分节
人类标注 [同意] / [修改] / [驳回]	子条目标注同步更新父节点 **子条目状态**; 父节点标注更新父节点 **确认状态**; 节点进入或移出AI可以处理节点的"待反馈处理"分节

规则	说明
R4	写回操作必须在本 Skill 的同一运行步骤内完成所有更新（索引表 + AI可以处理节点 + 子条目状态 + 文件头字段）。不允许"本次写节点、下次更新索引"
R5	索引表不存储行范围。AI 不依赖行号定位节点，全部通过 <code>grep -n '@{节点ID}'</code> 实时定位
R6	新增/删除节点后，必须同步更新索引表中的 节点数 和 待裁决 的计数值，精确计算
R7	AI可以处理节点的添加和移除必须精确匹配节点当前状态，仅当节点状态符合AI可以处理节点分节条件时加入，不符合时移出

编写 Skill 文档时，各 Skill 的 Step 布局总览表中 "写资产" 列应标注写回涉及的文件名。Step End 的增量清单中需注明本次写回是否更新了行索引。

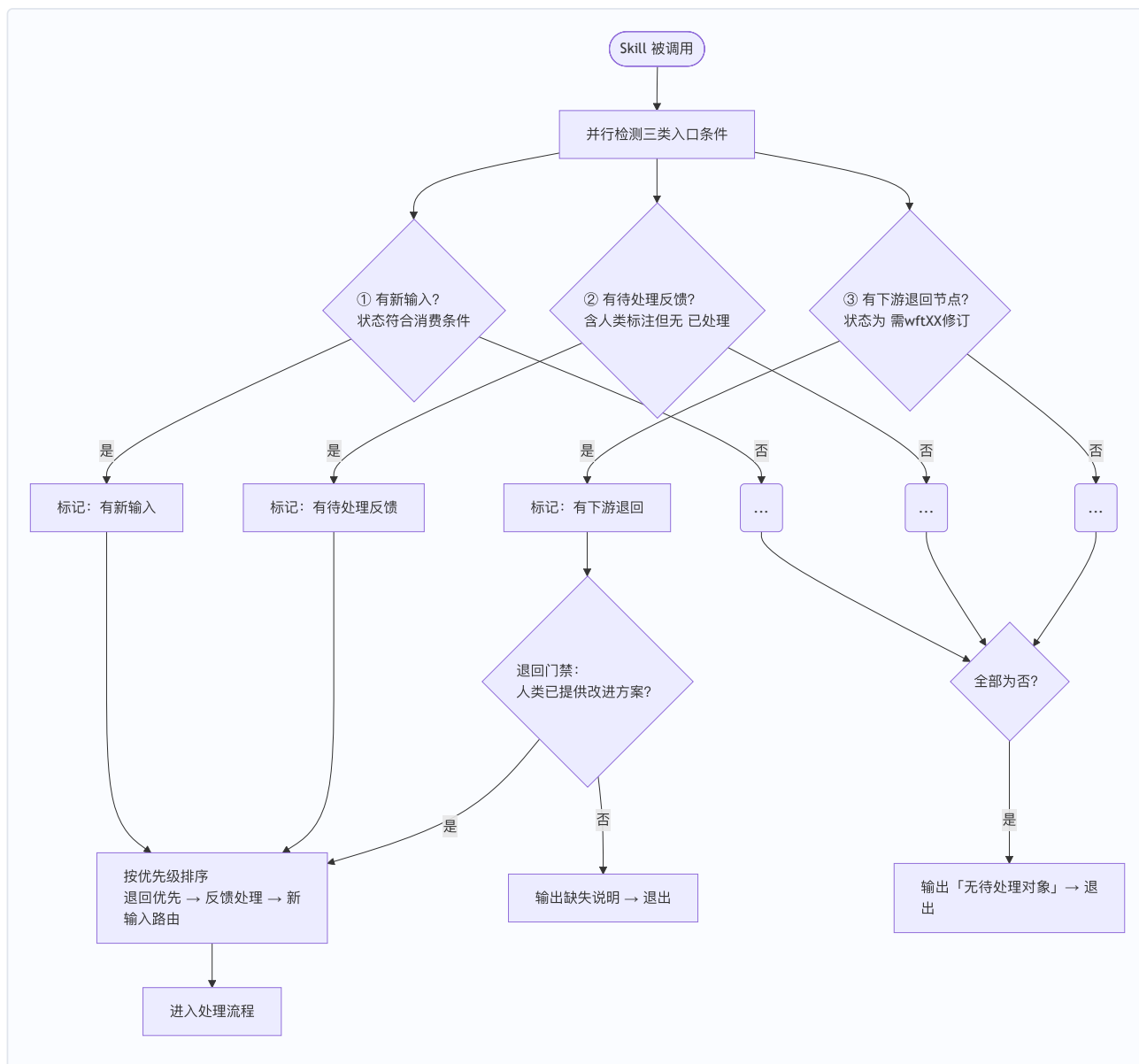
第三部分：Skill 运行时协议

适合谁： 需要理解 Skill 如何运行、AI 与人类如何交互的人。建议按 §A.4→§A.5→§A.6→§A.7 顺序阅读——四个协议依次构建：先进入 → 再处理反馈 → 状态随之流转 → 变更向下游传播。

A.4 入口条件与优先级（Step Start）

每个 Skill 的 Step Start 遵循统一的入口条件规范。本节定义四条通用规则，各 Skill 在此基础上按自身输入类型特化状态值和校验条件。

A.4.1 入口决策流程



实现细节：三类入口条件的节点清单不在此处逐一扫描判定，而是通过读取产品数据文档中的 ## AI可以处理节点 对应分节 (§11.9) 直接获取。AI可以处理节点由各 Skill 在写回时自动维护，减少 AI 在 Step Start 阶段的扫描量。

A.4.2 规则一：三种合法入口

#	入口条件	含义	示例（以 wft02-biz 为例）
①	有新输入	人类选中了本 Skill 的输入对象，状态符合消费条件	STR-F 节点状态为 可以分解分配 / 待补充分解分配
②	有待处理反馈	本 Skill 产出的方案中存在人类标注但尚未「已处理」的条目	BA 节点状态为 待确认方案，条目含「修改」未追加「已处理」
③	有下游退回节点	本 Skill 产出的节点被下游退回，等待修订	BA 节点状态为 需wft02修订

三种均不满足 → 输出"无待处理对象"→ **退出**。

A.4.3 规则二：退回节点的门禁

当入口条件③满足时（存在 需wftXX修订 的节点），需先检查人类是否已提供改进方案：

情况	动作
已提供改进方案	通过门禁，进入"退回优先"步骤
未提供改进方案	输出下游的缺失说明 + 提示提供改进方案 → 退出

改进方案的载体与判断标准："改进方案"的载体为下游 Skill 退回时附带的**缺失说明**——下游 Skill 在将节点状态写为待修订态时，须在节点末尾的变更影响声明中附加"缺失说明"小节，描述具体缺陷和建议修订方向。本 Skill 检测门禁时：

- 读取退回节点的缺失说明，检查其中是否包含人类标注（[修改] / [同意]）或下游 Skill 产出的具体改进指引（非仅"需修订"的概括性描述）
- 存在上述内容 → 门禁通过
- 仅含概括性退回原因（如"方案不完整，需修订"）且无人类标注 → 门禁不通过，提示人类先提供具体的改进方向

该门禁确保退回修订基于人类指引而非 AI 自行猜测。

A.4.4 规则三：优先执行顺序

同一次运行中多个入口条件同时满足时，按以下顺序执行：

退回优先 → 反馈处理 → 新输入路由

优先级	入口类型	原因
1（最高）	退回修订	退回修订和反馈处理都可能改变已有节点内容（边界/业务链/要素/产品路径）
2	反馈处理	新输入的匹配和设计必须基于修订后的最新内容
3（最低）	新输入路由	在上游变更和反馈处理完成后再处理新输入

A.4.5 规则四：各 Skill 的特化

各 Skill 在满足四条通用规则的前提下，按自身输入类型特化以下内容：

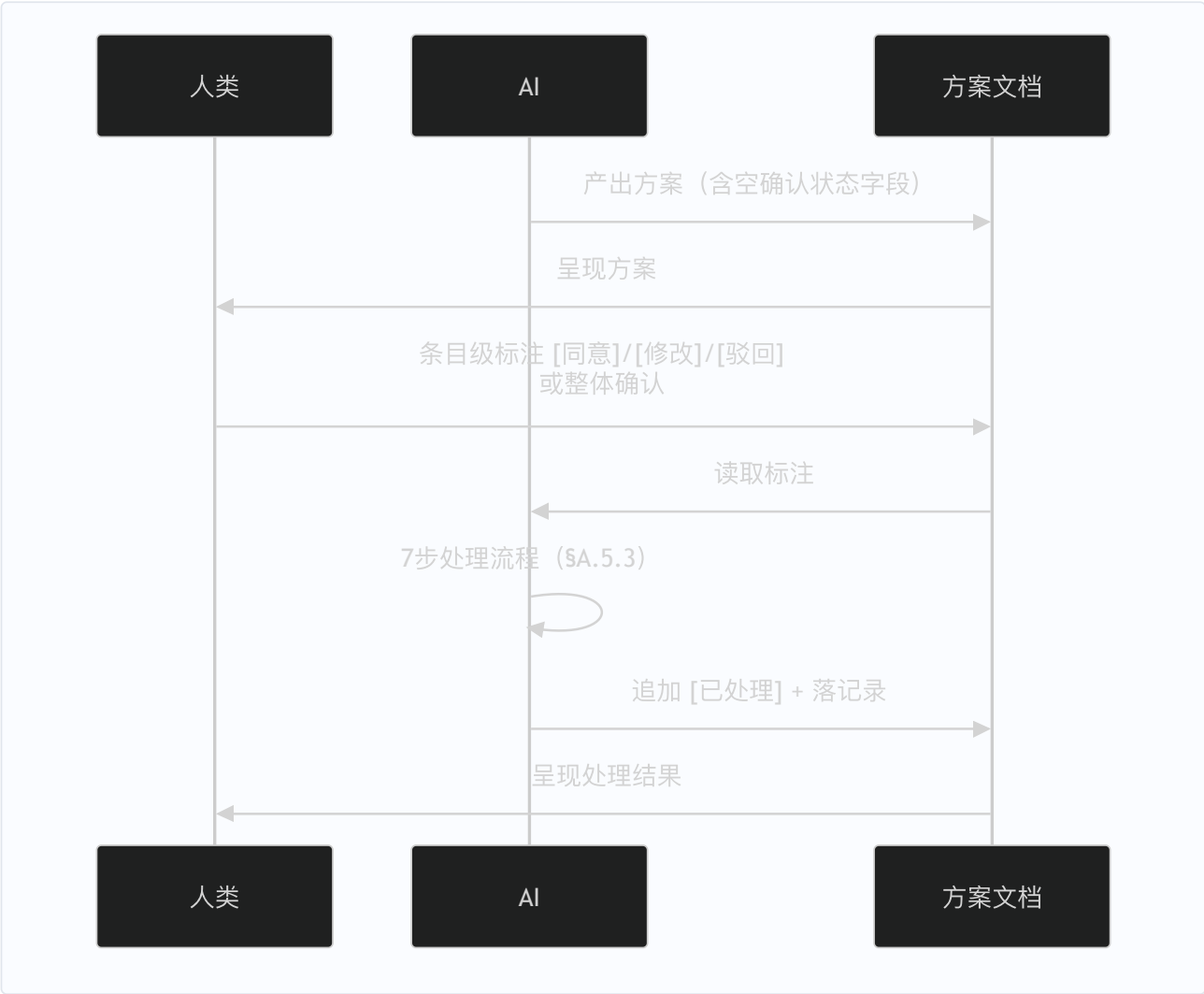
特化项	说明
输入对象类型	如 STR-F 节点、BA 节点、OR 条目
输入合法状态值	如 可以分解分配、待补充分解分配
退回节点状态值	如 需wft02修订
特有校验项	数量限制、跨域提示等（软提示，不由 AI 强制退出）

软提示 vs 硬退出：特有校验项（数量限制、跨域提示等）统一采用软提示——AI 输出超限/跨域提示后由人类选择继续或退出，不强制退出。逻辑上不可能的条件（状态不符、类型不匹配）仍为硬退出。

A.5 反馈交互协议

各 Skill 产出方案后，人类在方案的各项上直接标注，AI 处理后追加完成标记。以下为所有 Skill 统一的反馈处理流程。

A.5.1 交互流程总览



A.5.2 人类审阅规则

方式	操作	效力
条目级反馈	在条目旁标注 [同意] / [修改]：修改内容 / [驳回]：驳回理由	权威，优先级高于整体确认
整体确认	人类说「整体确认」→ AI 对无条目级标注的剩余条目写入 [同意] + [已处理]	快捷，仅补未标注条目
冲突消解	条目级 > 整体确认。两者并存时，条目级标注保留，整体确认仅补未标注的条目	—

已处理完的条目标注 [已处理]，后续轮次默认跳过。

A.5.3 AI 处理流程（7 步）

核心原则：反馈处理不是对人类标注的机械执行。AI 在反馈处理中与方案生成中同为设计者——先理解人类反馈在需求语境中的含义，再执行修订，最后检查修订后的方案是否仍满足其所承接的上层需求。

步骤	操作	详细说明
1. 扫描	遍历确认状态字段	筛选有人类标注（「同意」/「修改」/「驳回」）但无「已处理」的条目。已有「同意」+「已处理」的默认跳过——除非本轮新输入满足以下任一条件，此时先清除「已处理」（保留「同意」），使其回到待处理状态： • 新输入为该条目的直接上游节点（如新 OR 条目是某 BA 节点的来源） • 新输入与已有条目同属一个架构层级且边界重叠（如同属 PL6-A 的活动节点，新输入的加入改变了已有节点的范围或归属） • 新输入导致已有条目的定义被修订（变更影响声明标注为修订型或结构型） 不满足上述条件的已有条目继续跳过
2. 收集整体确认	处理「整体确认」指令	人类说「整体确认」→ 对无条目级标注的剩余条目写入「同意」+「已处理」
3. 逐条执行	按标注类型处理	见下方"分类型处理"表
4. 级联处理	处理变更的连锁影响	见下方"级联处理"表
5. 判定状态	决定方案是否可推进	全部条目含「同意」+「已处理」→ 推进至可推进态；任一条目含「修改」/「驳回」且无「需重设计」→ 保持 待确认方案；存在「需重设计」条目 → 保持 待确认方案，Step End 时提示该条目需进入设计步骤重做
6. 写回时机	确定何时写资产	仅有反馈处理（无新输入）→ 直接写回资产；同时有新输入 → 待设计步骤完成后统一写回
7. 落记录	写入处理记录	写入 ### 反馈处理记录 （含本轮各条目人类意见+来源+处理动作+需求级联影响+时间戳）

步骤 3 分类型处理：

标注	AI 动作（按顺序执行）
「同意」	不变，追加「已处理」
「修改」	① 理解修改在上层需求语境中的含义——要解决什么问题、是否改变了方案对上层需求的承接方式 → ② 执行修改 → ③ 检查修改后的方案是否仍满足上层需求（是否与上层约束冲突、是否引入需求缺口） → ④ 追加「已处理」
「驳回」	① 检查驳回造成的需求缺口——移除该对象后，上层需求的哪部分不再被承接 → ② 记录缺口说明，判断是否需要生成补充需求材料或退回上层重新处理 → ③ 移除被驳回对象及关联对齐结论 → ④ 追加「已处理」

步骤 4 级联处理（两级，按顺序执行）：

级联层级	触发条件	处理方式
需求级联检查（优先）	「修改」/「驳回」导致方案结构性变更（边界/拆分合并/引擎变更等）	重验受影响条目是否仍满足上层需求。若修改范围超出反馈处理可处理边界（如修改导致方案类型需重新判定），标记受影响条目为「需重设计」，交由设计步骤处理，不在反馈处理中自行重新设计
资产对齐级联（需求级联通过后）	方案变更影响资产引用	更新受影响资产的引用和对齐结论。资产映射由各 Skill 在第三章 Step 2 中定义

A.5.4 其他规则

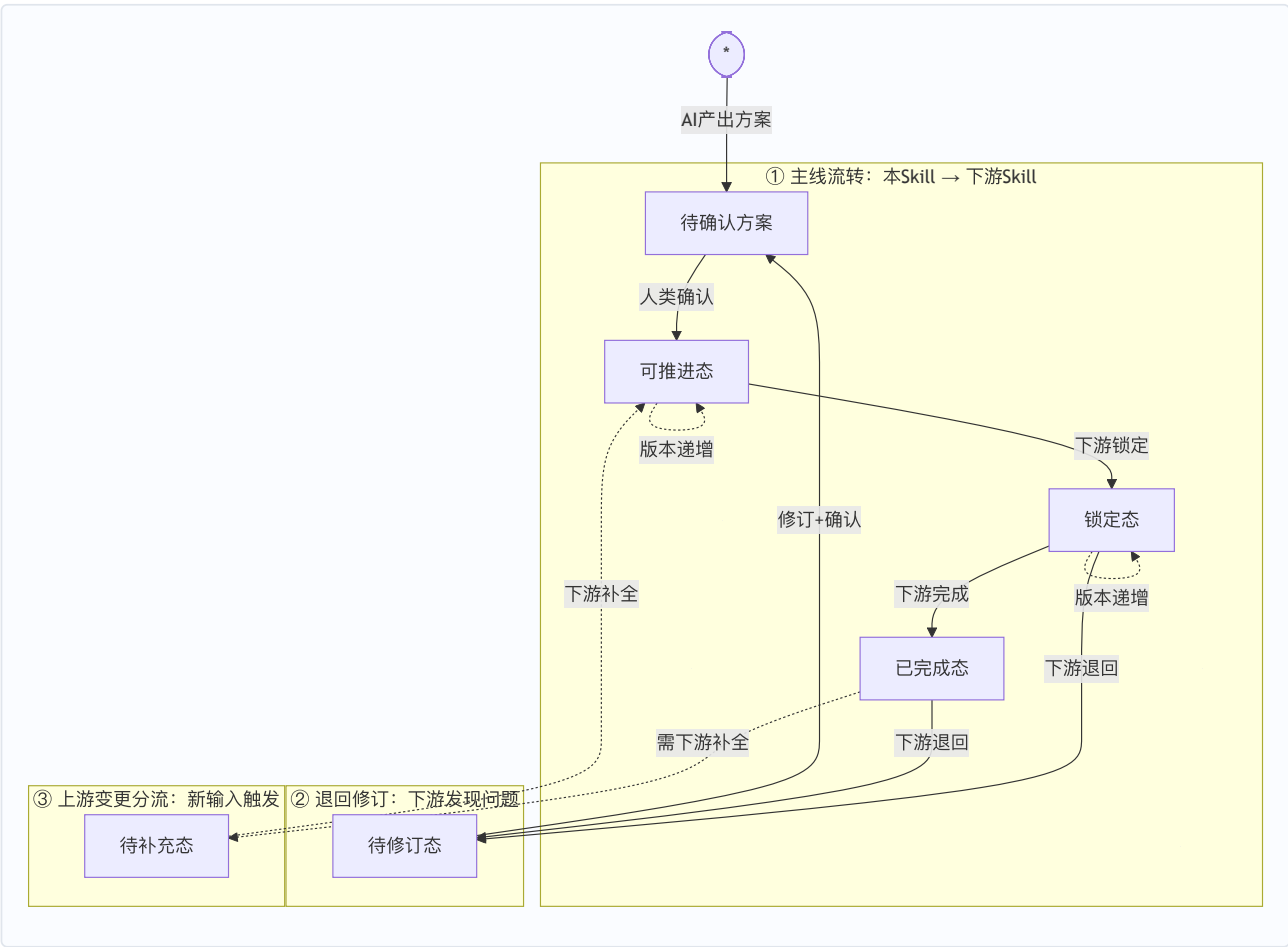
规则	说明
覆盖规则	人类改变主意时直接改写标注——新内容覆盖旧内容，「已处理」自然消失，条目回到待处理状态。处理的完整历史通过节点版本号 + 变更记录追溯
确认状态字段	AI 产出方案时为每个条目初始化确认状态字段（初始为空），人类在字段中直接标注，AI 处理后追加「已处理」。具体字段名、写入位置和条目粒度由各 Skill 在第三章 Step 2（方案反馈处理）的特化规则中定义并提供格式示例

反馈处理的特化参数（锚点需求、操作条目、推进目标、级联触发条件、资产对齐级联）由各 Skill 在第三章 Step 2 中定义。

A.6 节点生命周期模型

各路径 Skill 产出的节点遵循统一的生命周期模型：四态主线流转 + 上游变更三条分流 + 下游退回。各 Skill 的第二章（生命周期）引用本节并声明本 Skill 的状态名映射。

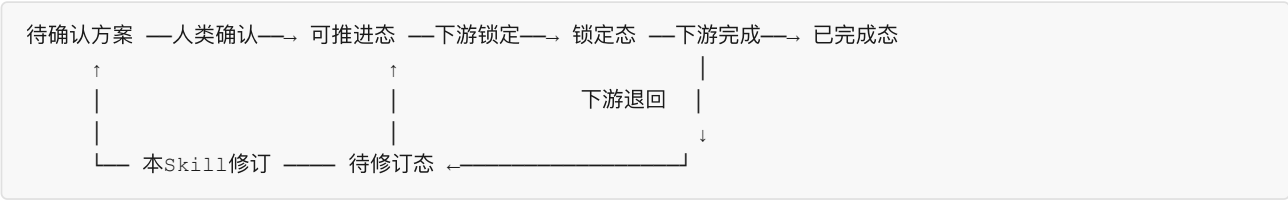
A.6.1 状态机总览



A.6.2 通用六态定义

通用态	含义	写入方	触发条件
待确认方案	方案已生成或修订，等待人类反馈	本 Skill	AI 产出方案 / 修订完成
可推进态	方案已确认，下游 Skill 可启动消费	本 Skill	人类确认全部条目
锁定态	下游 Skill 已锁定此节点，正在消费展开	下游 Skill	下游 Skill 启动消费
已完成态	下游消费完成，消费版本与当前版本一致	下游 Skill	下游产出完成
待修订态	下游发现缺陷，退回本 Skill 修订	下游 Skill	下游检出缺陷
待补充态	上游新输入导致下游产出不完整，需下游重新介入补全	上游 Skill	已完成态节点因上游新输入修订

A.6.3 主线流转



并行推进：人类确认后，资产写回（本 Skill）与节点消费（下游 Skill）两条线并行推进、互不阻塞。资产写回完成与否不改变节点状态。

A.6.4 上游变更分流

本 Skill 因新输入修订已有节点后，版本号递增，状态按下游消费进度分流：

原状态	修订后状态	含义
可推进态	可推进态（版本递增）	未被下游消费过，版本升级不影响下游
锁定态	锁定态（版本递增）	下游锁仍有效，重入时检测版本差，按变更类型增量处理
已完成态	待补充态	下游产出已不完整，需下游 Skill 重新介入补全

A.6.5 下游退回

下游 Skill 在消费展开中发现方案存在缺陷，将节点退回本 Skill 修订：

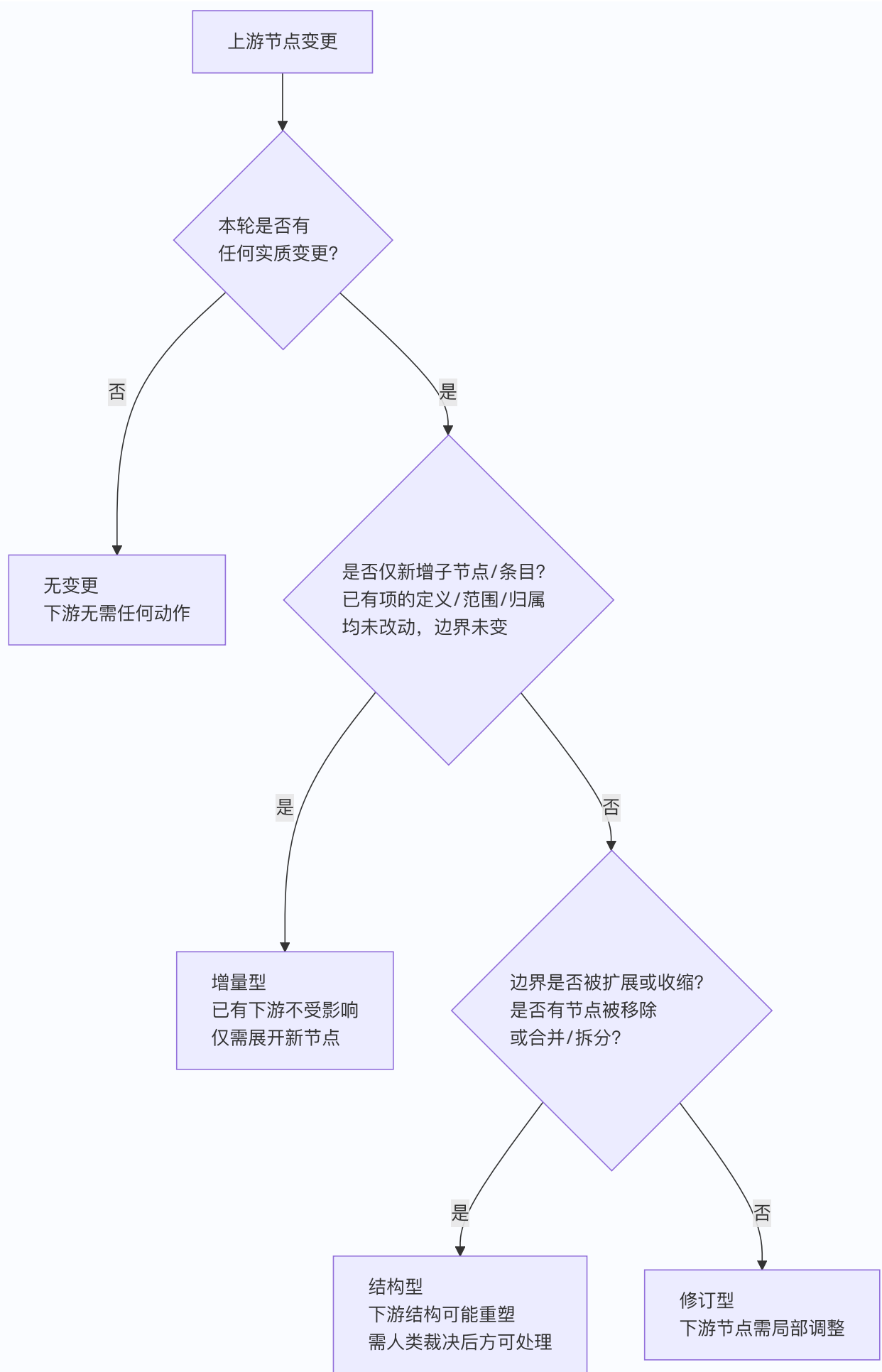
- 下游 Skill 将节点状态写为待修订态，附带缺失说明
- 本 Skill 重入时：读取下游缺失说明 → 按人类改进方案修订 → 转为待确认方案 → 人类确认后推进回可推进态

版本标记与消费追溯：节点初次生成时版本 = v_1 ；每次因上游新输入触发修订，版本号递增。下游 Skill 首次消费时记录 消费版本： $v_{\{N\}}$ ，重入时比对版本——若当前版本 > 已消费版本 → 触发变更感知，读取变更影响声明 (§A.7)，按变更类型处理。

A.7 变更传播协议

上游 Skill 产出节点，下游 Skill 消费后生成下游节点。当上游节点因新输入修订而变更时，已生成的下游节点面临同步问题。本协议定义变更分类、声明格式和消费追溯，使下游可增量响应而非全量重做。

A.7.1 变更判定流程



A.7.2 变更类型

变更类型	判定标准	对下游影响	下游动作
无变更	本轮新输入全部走来源确认或引用追加；无新增/改进/边界扩展/结构调整	下游无需任何动作	无
增量型	仅新增子节点/条目，已有项的定义/范围/归属均未改动；边界未变	已有下游节点不受影响	仅需展开新节点
修订型	已有项的定义/范围/归属被修改；或因新输入合并重识别导致属性变更；边界未变	对应下游节点需局部调整	修订受影响节点
结构型	边界检出扩展、子节点被移除或合并/拆分、或粒度不一致需拆分重组	下游结构可能重塑	需人类裁决后方可处理

判定优先级：结构型 > 修订型 > 增量型 > 无变更。

A.7.3 变更影响声明格式

通用模板（各 Skill 实例化时填充 {} 占位符）：

```
### 变更影响声明

{节点类型} 版本：v{N}（本次修订前为 v{N-1}）
本轮变更类型：无变更 / 增量型 / 修订型 / 结构型

变更项：
- [新增] / [变更] / [确认] / [需裁决] ...
  说明字段中记录细分原因

下游已消费此节点的 {下游节点类型}：
  <节点ID列表>（如无则写"尚未被下游消费"）

对已消费 {下游节点类型} 的影响：
  受影响节点：<节点ID + 受影响原因>
  不受影响节点：<节点ID>
  待新增 {下游节点类型}：<说明>

建议 {下游 Skill} 动作：
- 无变更：无需任何动作
- 增量型：增量展开新增内容即可，已有节点不动
- 修订型：修订受影响的节点 → 标记 [来自上游变更]
- 结构型：输出影响范围，等人类裁决后再处理
```

A.7.4 增量清单与变更影响声明的分工

维度	增量清单	变更影响声明
面向读者	人类	下游 Skill
目的	告知"本轮我做了什么，请逐项审阅"	告知"上游节点变了什么，你需要做什么"
写入位置	节点末尾（供人类参照 确认状态 字段逐条审阅）	节点末尾（独立小节）
生成时机	每次设计步骤产出方案时	仅修订已有节点时；新建节点写"首版，无变更"

各 Skill 特有的影响范围判定表（BA 变化→SR-F 受影响范围等）保留在各 Skill 文档中。

第四部分：快速参考

适合谁：已理解协议内容，需要速查清单的人。

A.8 快速参考

A.8.1 Skill 文档检查清单

写一个新的 Skill 文档时，按以下清单逐项检查：

#	检查项	参考
1	五章结构完整（职责定位 → 领域概念 → 操作流程 → AI标记 → 变更记录）	§A.2.1
2	第一章：目的与目标对齐 91 规范任务清单，含"负责/不负责"	§A.2.2
3	第二章：标题含末级节点名，只定义"是什么"不放操作流程	§A.2.3
4	第三章：Step 布局总览表在章首，Step Start 校验条件用表格	§A.2.4
5	第三章 Step 2：引用 §A.5 并声明本 Skill 特化参数	§A.5
6	第三章 Step End：使用三区模板	§A.3.2
7	第四章：引用 §A.3.1 + 本 Skill 增量清单示例	§A.3.1
8	生命周期：引用 §A.6 并声明状态名映射	§A.6
9	变更传播：引用 §A.7 并补充特有影响范围判定表	§A.7
10	Step Start 变更感知：引用 §A.3.3 步骤零（git diff H..HEAD）检测人类反馈	§A.3.3
11	读取协议：引用 §A.3.3——AI可以处理节点优先，索引导航备用，全部通过 grep 实时定位	§A.3.3

A.8.2 运行时协议速查

阶段	关键问题	答案位置
入口	Skill 什么时候被触发?	§A.4.2 规则一 (三种入口)
入口	多个条件同时满足怎么办?	§A.4.4 规则三 (退回 > 反馈 > 新输入)
入口	退回节点怎么处理?	§A.4.3 规则二 (先检查改进方案)
处理	人类 [修改] 后 AI 做什么?	§A.5.3 步骤 3 (理解→执行→检查→标记)
处理	[驳回] 后怎么处理需求缺口?	§A.5.3 步骤 3 (检查缺口→记录→移除→标记)
处理	修改触发了级联影响怎么办?	§A.5.3 步骤 4 (需求级联优先→资产对齐级联)
状态	节点有哪些状态?	§A.6.2 (六态)
状态	上游变更后节点状态怎么变?	§A.6.4 (按原状态三分流)
传播	怎么判断变更类型?	§A.7.1 (决策流程图) + §A.7.2 (四类型判定标准)
传播	变更影响怎么写?	§A.7.3 (声明模板)
入口	怎么知道哪些节点可以处理?	§11.7 AI可以处理节点 + §A.3.3 步骤零·变更感知
入口	人类直接在文档中写了反馈怎么办?	§A.3.3 步骤零 (git diff HEAD 变更检测)
入口	人类修改后提交了多次怎么办?	§A.3.3 步骤零 (git diff H..HEAD)
格式	AI 怎么标记本轮产出?	§A.3.1 (四类增量标记)
格式	Step End 输出什么?	§A.3.2 (三区模板)
维护	行范围 不对了怎么办?	索引表不存行范围, AI 通过 grep 实时定位节点 (§11.2)
维护	怎么知道上次 AI 运行到哪里?	文件头 <code>**HEAD @上次 AI 运行**</code> 字段 (§11.2)

各 Skill 的完整参数 (末级节点/输入对象/状态/下游/资产写回/补充材料) 定义在其 Skill 文档的第一章和第三章 Step Start 中。全量速查见 §10.6。